Lecture5a

September 29, 2025

1 CS5489 - Machine Learning

2 Lecture 5a - Neural Networks

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

- History
- Perceptron
- Multi-class logistic regression
- Multi-layer perceptron (MLP)

[1]:
```
# setup
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100  # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats

rbow = plt.get_cmap('rainbow')
```

4 Original idea

- *Perceptron*
  - Warren McCulloch and Walter Pitts (1943), Rosenblatt (1957)
  - Simulate a neuron in the brain
    1) take binary inputs (input from nearby neurons)
    2) multiply by weights (synapses, dendrites)
    3) sum and threshold to get binary output (output axon)
  - Train weights from data.
- Multiple outputs handled by using multiple perceptrons
- **Problem:**

  - linear classifier, can't solve harder problems

5 Multi-layer Perceptron

- Add *hidden* layers between input and output neurons
  - each layer extracts some features from the previous layers
  - can represent complex non-linear functions
  - train weights using *backpropagation* algorithm. (1970-80s)
  - (now called a *neural network*)
- **Problem:**
  - difficult to train.
  - sensitive to initialization.
  - computationally expensive (at the time).

6 Decline in the 1990s

- Because of those problems, NN became less popular in the 1990s
  - Support vector machines (SVM) had good accuracy
    - easy to use - only one global optimum.
    - learning is not sensitive to initialization.
    - theory about generalization guarantees.
  - Not a lot of data, so kernel methods were still okay.

7 Deep learning

- There was a resurgence in NN in the 2000s, due to a number of factors:
  - improvements in network architecture
    - developed nodes that are easier to train
  - better training algorithms
    - better ways to prevent overfitting
    - better initialization methods
  - faster computers
    - massively parallel GPUs (Thanks to gamers!)
  - more labeled data
    - from Internet
    - crowd-sourcing for labeling data (Amazon Turk)
- We can train NN with more and more layers ⇒ Deep Learning

[2]:
```
def drawplane(w, b=None, c=None, wlabel=None, poscol=None, negcol=None, lw=2,␣
↪col='k', ls='-', label=None):
    # w^Tx + b = 0
    # w0 x0 + w1 x1 + b = 0
    # x1 = -w0/w1 x0 - b / w1

    # OR
```

```
    # w^T (x-c) = 0 = w^Tx - w^Tc  --> b = -w^Tc
    if c is not None:
        b = -sum(w*c)

    # the line
    if (abs(w[0])>abs(w[1])):   # vertical line
        x0 = array([-30,30])
        x1 = -w[0]/w[1] * x0 - b / w[1]
    else:                       # horizontal line
        x1 = array([-30,30])
        x0 = -w[1]/w[0] * x1 - b / w[0]

    if (poscol) or (negcol):
        polyx = [x0[0], x0[-1], x0[-1], x0[0]]
        polyy = [x1[0], x1[-1], x1[0], x1[0]]
        f = sum(w*[polyx[2], polyy[2]])+b
        if (f>0) and (poscol):
            plt.fill(polyx, polyy, poscol, alpha=0.2)
        if (f<0) and (negcol):
            plt.fill(polyx, polyy, negcol, alpha=0.2)

        polyx = [x0[0], x0[-1], x0[0], x0[0]]
        polyy = [x1[0], x1[-1], x1[-1], x1[0]]
        f = sum(w*[polyx[2], polyy[2]])+b
        if (f>0) and (poscol):
            plt.fill(polyx, polyy, poscol, alpha=0.2)
        if (f<0) and (negcol):
            plt.fill(polyx, polyy, negcol, alpha=0.2)

    # plot line
    plt.plot(x0, x1, col+ls, lw=lw, label=label)

    # the w
    if (wlabel is not None):
        xp = array([0, -b/w[1]])
        xpw = xp+w
        plt.arrow(xp[0], xp[1], w[0], w[1], head_width=0.4,  width=0.01,␣
 ↪linestyle=ls, edgecolor=col, facecolor=col)
        plt.text(xpw[0]-0.5, xpw[1], wlabel)

def plot_perceptron(wb, X, Y, axbox, curn=None, wb_old=None):
    if wb_old is None:
        label1 = 'w'
    else:
        label1 = 'new w'
        label2 = 'old w'
    drawplane(wb[0], wb[1], wlabel="", label=label1)
```

```
    plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
    if curn is not None:
        plt.plot(X[curn,0], X[curn,1], 'ko', fillstyle='none',␣
 ↪markeredgewidth=2, label='selected point')
    if wb_old is not None:
        drawplane(wb_old[0], wb_old[1], lw=1, col='r', ls="--", label=label2)
    plt.grid(True)
    plt.axis(axbox)
```

[3]:
```
# generate random data
X,Y = datasets.make_blobs(n_samples=50,
         centers=array([[2,5], [-2,-3.2]]), cluster_std=2, n_features=2,
         random_state=4487)
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#FFFFFF", "#00FF00"])

lsdatafig = plt.figure()
plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
plt.xlabel('$x_1$'); plt.ylabel('$x_2$')
axbox = [-10,10,-10,10]
plt.axis(axbox); plt.grid(True);
plt.close()
```

8 Outline

- History
- **Perceptron**
- Multi-class logistic regression
- Multi-layer perceptron (MLP)

9 Perceptron

- Model a single neuron
  - input $\mathbf{x} \in \mathbb{R}^d$ is a d-dim vector
  - apply a weight to the inputs
  - sum and threshold to get the output $y \in \{+1, -1\}$
- Formally,
  - $y = f(\sum_{j=0}^{d} w_j x_j) = f(\mathbf{w}^T \mathbf{x})$
  - $\mathbf{w}$ is the weight vector.
  - $f(a)$ is the activation function
    - $f(a) = \begin{cases} 1, & a \geq 0 \\ -1, & \text{otherwise} \end{cases}$

10 Perceptron training criteria

- Train the perceptron on data $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$
- Only look at the points that are misclassified.
  - Loss is based on how badly misclassified

$$E(\mathbf{w}) = \sum_{i=1}^{N} \begin{cases} -y_i \mathbf{w}^T \mathbf{x}_i, & \mathbf{x}_i \text{ is misclassified} \\ 0, & \text{otherwise} \end{cases}$$

- Minimize the loss: $\mathbf{w}^* = \text{argmin}_{\mathbf{w}} E(\mathbf{w})$

11 Perceptron Loss Function

- Define $z_i = y_i \mathbf{w}^T \mathbf{x}_i$,
- The loss function is

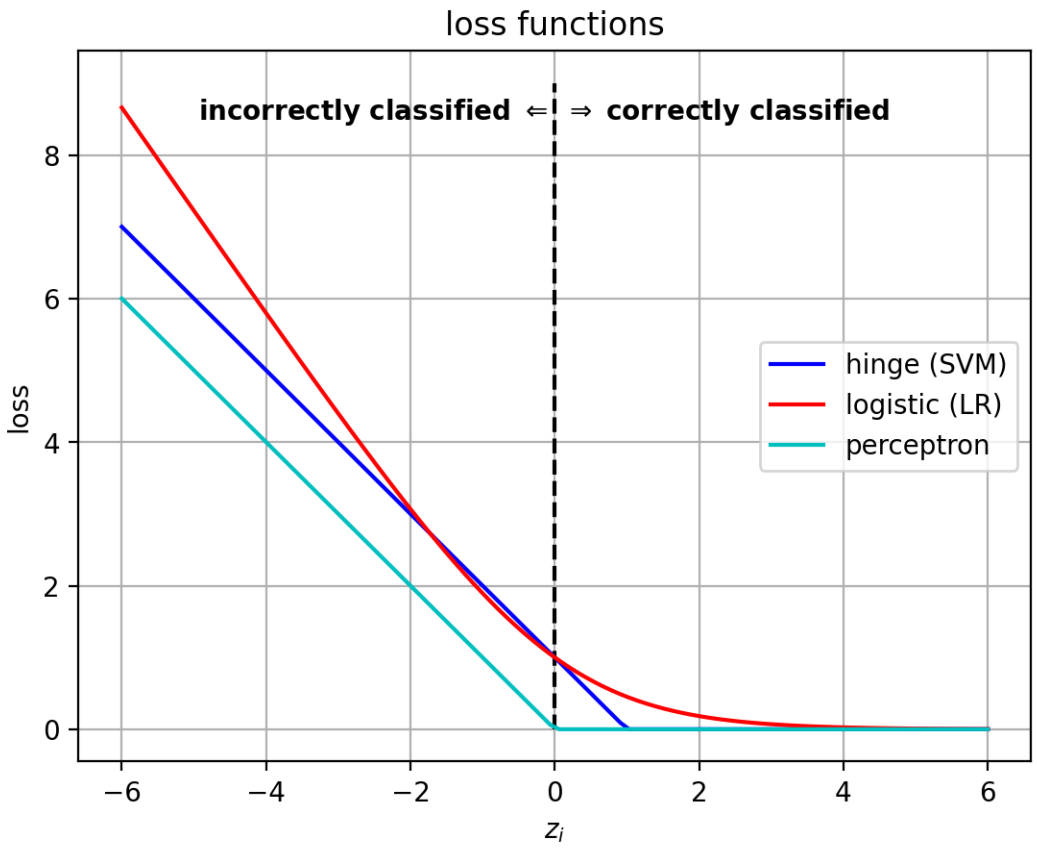
$$L(z_i) = \max(0, -z_i)$$

[4]:
```
z = linspace(-6,6,100)
logloss = log(1+exp(-z)) / log(2)
hingeloss = maximum(0, 1-z)
percloss = maximum(0,-z)
lossfig = plt.figure()

plt.plot([0,0], [0,9], 'k--')
plt.text(0,8.5, "incorrectly classified $\\Leftarrow$ ", ha='right',␣
 ↪weight='bold')
plt.text(0,8.5, " $\Rightarrow$ correctly classified", ha='left', weight='bold')

plt.plot(z,hingeloss, 'b-', label='hinge (SVM)')
plt.plot(z,logloss, 'r-', label='logistic (LR)')
plt.plot(z,percloss, 'c-', label='perceptron')
plt.grid(True)
plt.xlabel('$z_i$');
plt.ylabel('loss')
plt.legend(loc='right')
plt.title('loss functions')
plt.close()
```

```
<>:9: SyntaxWarning: invalid escape sequence '\R'
<>:9: SyntaxWarning: invalid escape sequence '\R'
/var/folders/v0/s4761j_n3z7_f651dfw62m5w0000gn/T/ipykernel_42519/712462370.py:9:
SyntaxWarning: invalid escape sequence '\R'
  plt.text(0,8.5, " $\Rightarrow$ correctly classified", ha='left',
weight='bold')
```

[5]:
```
lossfig
```

[5]:



12 Training algorithm

- We can learn the model by applying gradient descent.
  - Move $\mathbf{w}$ in the direction to decrease the loss $E(\mathbf{w})$.
  - Gradient descent update: $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{d}{d\mathbf{w}} E(\mathbf{w})$
    - η is the learning rate for gradient descent

- Computers were slow back then…
- Solution: only look at one data point at a time and use gradient descent.
  - loss of a misclassified point $\mathbf{x}_i$: $E_i(\mathbf{w}) = -y_i \mathbf{w}^T \mathbf{x}_i$
  - Gradient: $\frac{d}{d\mathbf{w}} E_i(\mathbf{w}) = -y_i \mathbf{x}_i$

- **Perceptron Algorithm**
  - For each point $\mathbf{x}_i$,
    - If the point $\mathbf{x}_i$ is misclassified,
      - Update weights: $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$
  - Repeat until no more points are misclassified

- **Notes:**
  - The effect of the update step is to rotate $\mathbf{w}$ towards the misclassified point $\mathbf{x}_i$.
  - This is called *Stochastic Gradient Descent*.

    * useful because we only need to look at a little bit of data at a time.
    * less computing/memory requirement in each iteration.

[6]:
```
def run_perceptron(w_init, X, Y, iterfigs=False):
    w = w_init.copy()
    eta = 1.0

    figs=[]

    for t in range(10):
        # cycle through data points
        haserror = False
        for n in range(len(X)):
            yx = Y[n]*X[n]
            zp = sum(w*yx)
            if (zp<0):
                w_old = w.copy()
                w += eta*yx
                haserror = True
                if iterfigs:
                    figs.append(plt.figure(figsize=(10,4.5)))
                    plt.subplot(1,2,1)
                    plot_perceptron((w_old,0), X, Y, axbox, curn=n)
                    plt.legend(loc='best')
                    plt.subplot(1,2,2)
                    plot_perceptron((w,0), X, Y, axbox, curn=n,␣
 ↪wb_old=(w_old,0))
                    plt.legend(loc='best')
                    plt.close()
        print(haserror)
        if not haserror:
            break

    if iterfigs:
        return (w, figs)
    else:
        return w
```

[7]:
```
w,figs = run_perceptron(array([1.0, 0.01]), X, Y, iterfigs=True)
```

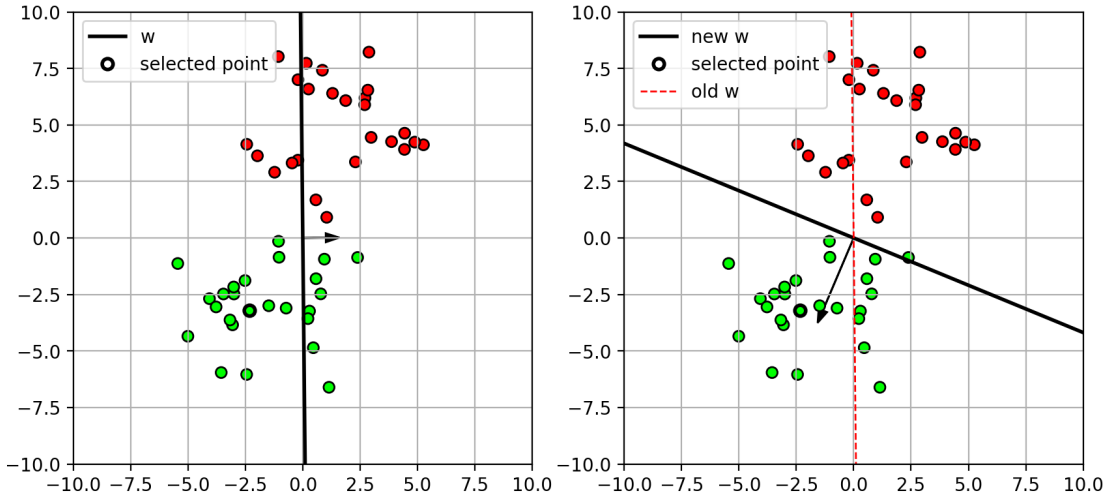
```
True
True
False
```

13 Example

- Iteration 1
  - **w** rotates towards the misclassified point (bold circle)

[8]:
```
figs[0]
```

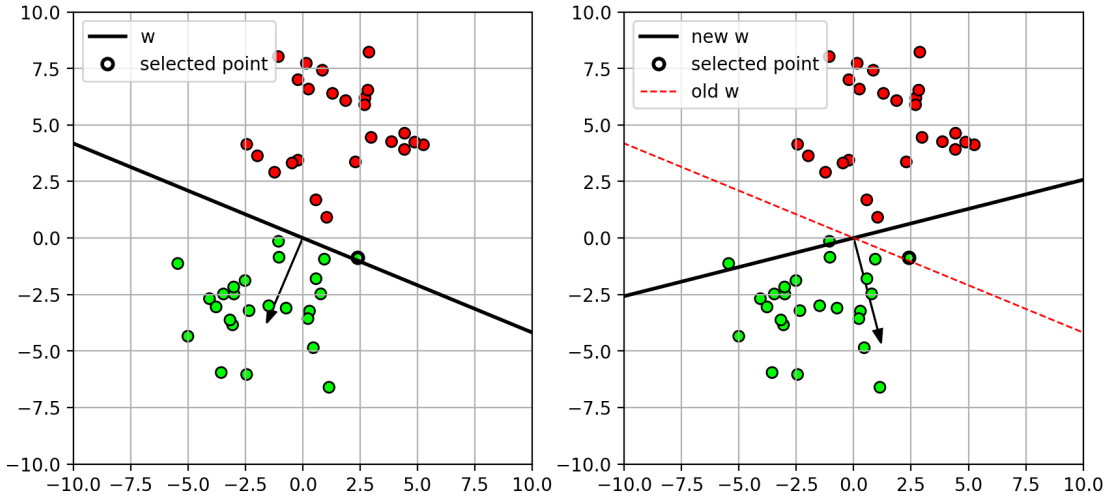
[8]:



- Iteration 2

[9]:
```
figs[1]
```

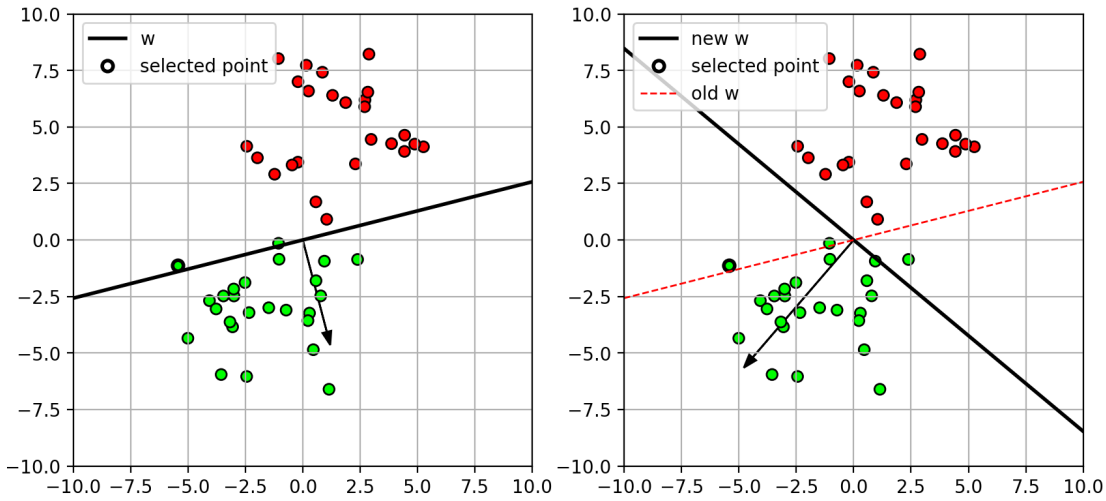
[9]:



- Iteration 3

[10]:
```
figs[2]
```

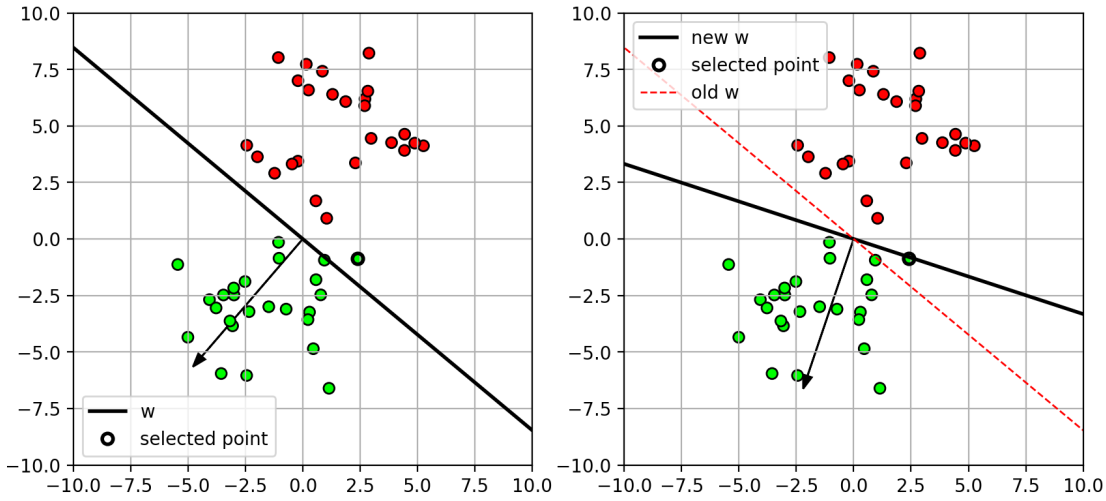
[10]:



- Iteration 4
  - No more errors

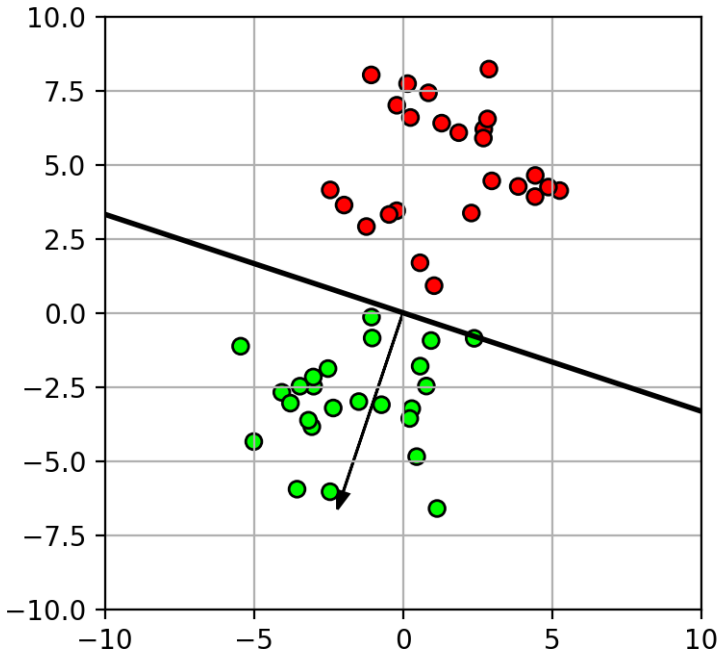
[11]:
```
figs[3]
```

[11]:



- Final classifier

[12]:
```
plt.figure(figsize=(4,4))
plot_perceptron((w,0),X,Y,axbox)
```



14 Perceptron Algorithm

- Fails to converge if data is not linearly separable
- Rosenblatt proved that the algorithm will converge if the data is linearly separable.
  - the number of iterations is inversely proportional to the separation (margin) between classes.
  - *This was one of the first machine learning results!*
- Different initializations can yield different weights
  - There are multiple decision boundaries with 0 loss.

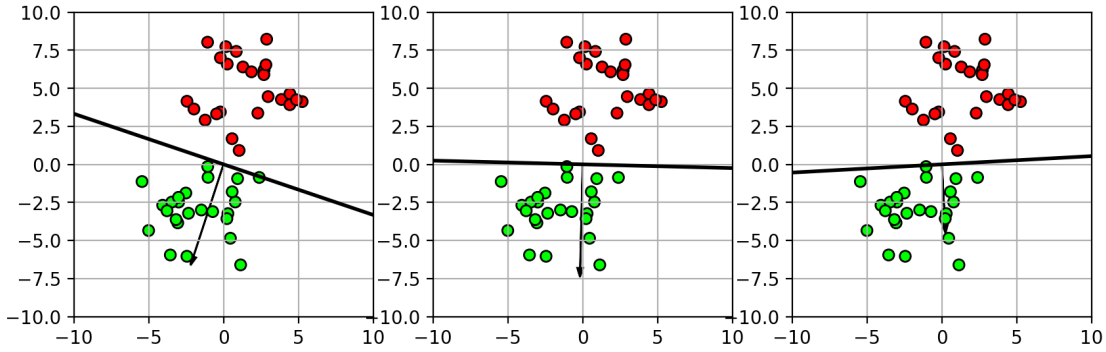
[13]:
```
ni = random.permutation(arange(len(X)))
w2 = run_perceptron(array([1.0, 0.01]), X[ni], Y[ni])
ni = random.permutation(arange(len(X)))
w3 = run_perceptron(array([1.0, 0.01]), X[ni], Y[ni])
pfig = plt.figure(figsize=(10,3))
plt.subplot(1,3,1)
plot_perceptron((w,0),X,Y,axbox)
plt.subplot(1,3,2)
plot_perceptron((w2,0),X,Y,axbox)
plt.subplot(1,3,3)
plot_perceptron((w3,0),X,Y,axbox)
plt.close()
```

```
True
False
```

```
True
True
False
```

[14]:
```
pfig
```

[14]:



15 Outline

- History
- Perceptron
- **Multi-class logistic regression**
- Multi-layer perceptron (MLP)

16 Revisiting Multiclass logistic regression

- Consider a multi-class classification problem with C classes
  - class labels $y \in \{1, \cdots, C\}$
  - equivalently, class vectors:

$$\mathbf{y} \in \{ \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \cdots, \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix} \} = \{\mathbf{e}_1, \mathbf{e}_2, \cdots, \mathbf{e}_C\}$$

    * $\mathbf{e}_j$ is the canonical vector.

17 Linear functions

- Construct C linear functions, one for each class
  - $g_j(\mathbf{x}) = \mathbf{w}_j^T \mathbf{x}$, for $j = \{1, \cdots, C\}$
  - $\mathbf{w}_j$ is the weight vector for the j-th class.
  - (to reduce clutter, we implicitly include the bias term)
- Combine into a vector-valued function ($\mathbb{R}^C$):

- $\mathbf{g}(\mathbf{x}) = \begin{bmatrix} g_1(\mathbf{x}) \\ \vdots \\ g_C(\mathbf{x}) \end{bmatrix} = \mathbf{W}^T\mathbf{x}$,
- Weight matrix: $\mathbf{W} = [\mathbf{w}_1, \cdots, \mathbf{w}_C]$

18 Mapping to probabilities

- output of $\mathbf{g}(\mathbf{x})$ is a real vector in $\mathbb{R}^C$.
- How to map it to set of class probabilities?
  - require $0 \leq p(y = j|\mathbf{x}) \leq 1$
  - and $\sum_{j=1}^{C} p(y = j|\mathbf{x}) = 1$.

19 Softmax function

- Given a real vector $\mathbf{a} \in \mathbb{R}^C$
- Let $s_j(\mathbf{a}) = \frac{\exp(a_j)}{\sum_{k=1}^{C} \exp(a_k)}$
  - if $a_j \gg a_i$, then the exponent will cause $s_j(\mathbf{a}) \to 1$.
  - denominator ensures $\sum_{j=1}^{C} s_j(\mathbf{a}) = 1$.
- Let $\mathbf{s}(\mathbf{a}) = [s_1(\mathbf{a}) \cdots s_C(\mathbf{a})]^T$
  - the output vector is ~1 in the dimension of $\mathbf{a}$ with largest value, and 0 elsewhere.
  - called the **softmax** function ("soft" because the values can be between 0 and 1)

20 Mapping to probabilities

- Define the probability of the j-th class $p(y = j|\mathbf{x})$ as:
  - $p(y = j|\mathbf{x}) = f_j(\mathbf{x}) = s_j(\mathbf{g}(\mathbf{x})) = \frac{\exp(g_j(\mathbf{x}))}{\sum_{k=1}^{C} \exp(g_k(\mathbf{x}))}$
    - if $g_j(\mathbf{x}) \gg g_i(\mathbf{x})$, then the exponent will cause numerator to be very large, and thus $p(y = j|\mathbf{x}) \to 1$.
    - the class with largest response $g_j(\mathbf{x})$ will have highest probability.
    - denominator ensures probabilities sum to 1 over classes.
- Finally, define the posterior probability vector:

$$\begin{bmatrix} p(y = 1|\mathbf{x}) \\ \vdots \\ p(y = C|\mathbf{x}) \end{bmatrix} = \mathbf{f}(\mathbf{x}) = \mathbf{s}(\mathbf{g}(\mathbf{x})))$$

21 Example

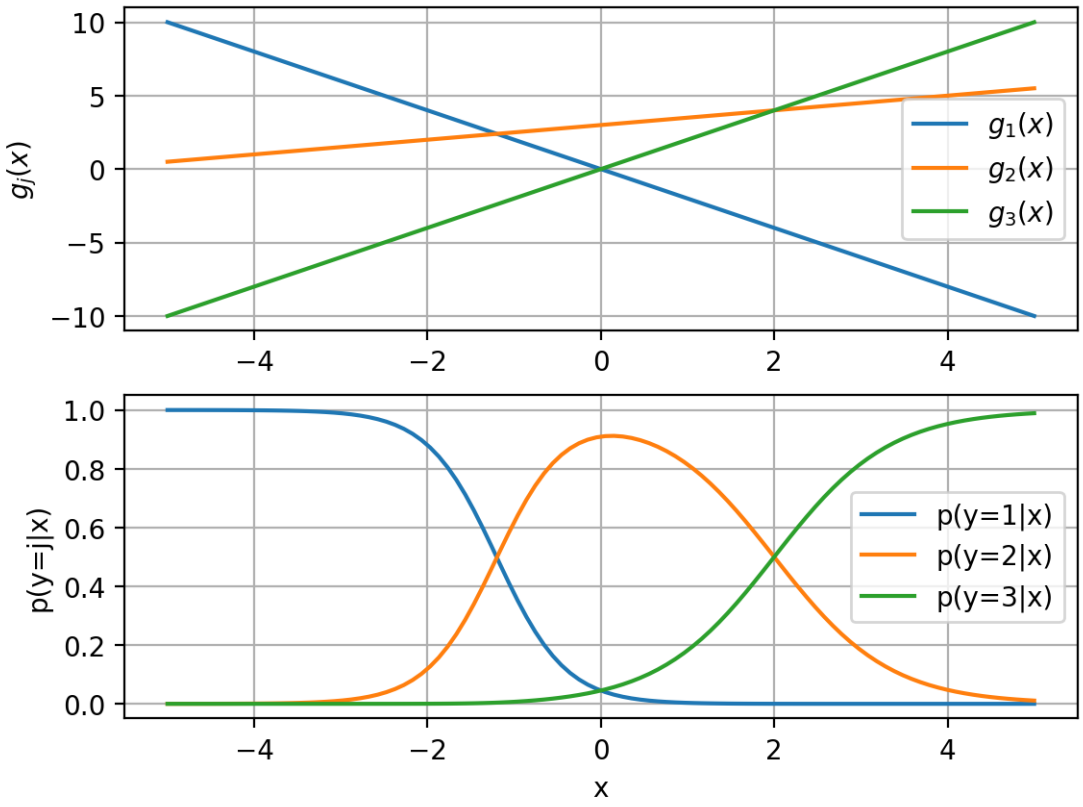
- linear functions and mapped probabilities

[15]:
```
x = linspace(-5,5,100)
w = array([-2, 0.5, 2])
b = array([0, 3, 0])

g = x[:,newaxis]*w+b
```

```
s = exp(g)
s /= sum(s,axis=1)[:,newaxis]

sfig = plt.figure()
plt.subplot(2,1,1)
plt.plot(x, g[:,0], label='$g_1(x)$')
plt.plot(x, g[:,1], label='$g_2(x)$')
plt.plot(x, g[:,2], label='$g_3(x)$')
plt.legend()
#plt.title('linear function')
plt.ylabel('$g_j(x)$')
plt.grid(True)
plt.subplot(2,1,2)
plt.plot(x, s[:,0], label='p(y=1|x)')
plt.plot(x, s[:,1], label='p(y=2|x)')
plt.plot(x, s[:,2], label='p(y=3|x)')
plt.legend()
plt.grid(True)
plt.xlabel('x')
plt.ylabel('p(y=j|x)')
#plt.title('probability')
plt.close()
```

[16]:
```
sfig
```

[16]:



22 Learning with MLE

- let $\mathbf{y}$ be the *class vector* representation of the class
  - i.e. $y_j = 1$ indicates class $y = j$, and 0 otherwise.
- **Log-likelihood function**
  - categorical distribution

$$\begin{aligned}\log p(\mathbf{y}|\mathbf{x}) &= \log \prod_{j=1}^{C} f_j(\mathbf{x})^{y_j} \\ &= \sum_{j=1}^{C} y_j \log f_j(\mathbf{x}) = \mathbf{y}^T \log \mathbf{f}(\mathbf{x})\end{aligned}$$

    - Note: the log of a vector is element-wise log.

- Maximum Likelihood Estimation (MLE)
  - Let $D = \{(\mathbf{y}_i, \mathbf{x}_i)\}$ be the training set.
  - MLE goal:

$$\begin{aligned}\mathbf{W}^* &= \underset{\mathbf{W}}{\operatorname{argmax}} \sum_{i=1}^{N} \log p(\mathbf{y}_i|\mathbf{x}_i) \\ &= \underset{\mathbf{W}}{\operatorname{argmax}} \sum_{i=1}^{N} \mathbf{y}_i^T \log \mathbf{f}(\mathbf{x}_i) \\ &= \underset{\mathbf{W}}{\operatorname{argmax}} \sum_{i=1}^{N} \sum_{j=1}^{C} y_{ij} \log f_j(\mathbf{x}_i)\end{aligned}$$

- Equivalently, turn maximization problem into minimization

$$\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^{N} \left\{ -\sum_{j=1}^{C} y_{ij} \log f_j(\mathbf{x}_i) \right\} = \sum_{i=1}^{N} L(\mathbf{y}_i, \mathbf{f})$$

  - Called the **cross-entropy loss** between ground-truth $\mathbf{y}_i$ and prediction $\mathbf{f}(\mathbf{x}_i)$
    - $L(\mathbf{y}, \mathbf{f}) = -\sum_{j=1}^{C} y_j \log f_j(\mathbf{x})$

23 How to optimize?

- Use gradient descent:
  - $\mathbf{W}^{(t)} = \mathbf{W}^{(t-1)} - \eta \left.\frac{dL}{d\mathbf{W}}\right|_{\mathbf{W}^{(t-1)}}$
    - gradient evaluted at current parameters $\mathbf{W}^{(t-1)}$.
- How do we compute the gradient?
  - We have a composition of functions:
    - $\mathbf{g}(\mathbf{x}) = \mathbf{W}^T\mathbf{x}$
    - $\mathbf{f}(\mathbf{x}) = \mathbf{s}(\mathbf{g}(\mathbf{x}))$
    - $L(\mathbf{y}, \mathbf{f}) = -\mathbf{y}^T \log \mathbf{f}(\mathbf{x})$

- Use the chain rule!
  - in one-dimension:
    - suppose $f(x) = s(g(x))$
    - by the chain rule: $\frac{df}{dx} = \frac{df}{dg}\frac{dg}{dx}$
  - our case is more complicated because of vector-valued functions.

24 Applying the Chain rule

- Work backwards to compute gradient
- Gradient of loss wrt **f**:

$$\frac{dL}{d\mathbf{f}} = \begin{bmatrix} \frac{dL}{df_1} \\ \vdots \\ \frac{dL}{df_C} \end{bmatrix}$$

- Gradient of loss wrt **g**:
  - First, look at individual g_j
    - changes in g_j affect all f_k, so sum over derivatives of f_k.

$$\frac{dL}{dg_j} = \sum_{k=1}^{C} \frac{dL}{df_k}\frac{df_k}{dg_j} = \frac{d\mathbf{f}^T}{dg_j}\frac{dL}{d\mathbf{f}}$$

    - where $\frac{d\mathbf{f}^T}{dg_j} = [\frac{df_1}{dg_j} \dots \frac{df_C}{dg_j}]$.

- Gradient of loss wrt **g**:
  - For all linear functions **g**

$$\frac{dL}{d\mathbf{g}} = \begin{bmatrix} \frac{dL}{dg_1} \\ \vdots \\ \frac{dL}{dg_C} \end{bmatrix} = \begin{bmatrix} \frac{d\mathbf{f}^T}{dg_1}\frac{dL}{d\mathbf{f}} \\ \vdots \\ \frac{d\mathbf{f}^T}{dg_C}\frac{dL}{d\mathbf{f}} \end{bmatrix} = \frac{d\mathbf{f}^T}{d\mathbf{g}}\frac{dL}{d\mathbf{f}}$$

    - where the Jacobian (transpose) is:

$$\frac{d\mathbf{f}^T}{d\mathbf{g}} = \begin{bmatrix} \frac{df_1}{dg_1} & \dots & \frac{df_C}{dg_1} \\ \vdots & \ddots & \vdots \\ \frac{df_1}{dg_C} & \dots & \frac{df_C}{dg_C} \end{bmatrix}$$

      - (how each input dimension of affects each output dimension of $\mathbf{f}(\mathbf{g})$)

- Gradient of loss wrt $\mathbf{w}_j$:
  - $\frac{dL}{d\mathbf{w}_j} = \sum_{k=1}^{C} \frac{dg_k}{d\mathbf{w}_j}\frac{dL}{dg_k} = \frac{d\mathbf{g}^T}{d\mathbf{w}_j}\frac{dL}{d\mathbf{g}}$
  - since weight vector $\mathbf{w}_j$ only appears in g_j
    - $\frac{dL}{d\mathbf{w}_j} = \frac{dg_j}{d\mathbf{w}_j}\frac{dL}{dg_j}$

25 Summary:

- Chain rule:
  - compute the gradient by working backwards from **f** to $\mathbf{w}_j$
    1) Gradient of loss wrt **f**: $\frac{dL}{d\mathbf{f}}$

2) Gradient of loss wrt $\mathbf{g}$: $\frac{dL}{d\mathbf{g}} = \frac{d\mathbf{f}^T}{d\mathbf{g}} \frac{dL}{d\mathbf{f}}$

3) Gradient of loss wrt $\mathbf{w}_j$: $\frac{dL}{d\mathbf{w}_j} = \frac{d\mathbf{g}^T}{d\mathbf{w}_j} \frac{dL}{d\mathbf{g}}$

- Final result after some derivation of gradients:
  - $\frac{dL}{d\mathbf{w}_j} = \mathbf{x}(f_j(\mathbf{x}) - y_j)$

26 Example

[17]:
```
# load iris data each row is (petal length, sepal width, class)
irisdata = loadtxt('iris3.csv', delimiter=',', skiprows=1)

X = irisdata[:,0:2]  # the first two columns are features (petal length, sepal␣
 ↪width)
Y = irisdata[:,2]    # the third column is the class label (setosa=0,␣
 ↪versicolor=1, virginica=2)

print(X.shape)

# randomly split data into 50% train and 50% test set
trainX, testX, trainY, testY = \
  model_selection.train_test_split(X, Y,
  train_size=0.5, test_size=0.5, random_state=4487)

print(trainX.shape)
print(testX.shape)
```

```
(150, 2)
(75, 2)
(75, 2)
```

[18]:
```
# learn logistic regression classifier
mlogreg = linear_model.LogisticRegression(C=10,
            multi_class='multinomial', solver='lbfgs')
            # use multi-class and corresponding solver
mlogreg.fit(trainX, trainY)

# now contains 3 hyperplanes and 3 bias terms (one for each class)
print("w=", mlogreg.coef_)
print("b=", mlogreg.intercept_)

# predict from the model
predY = mlogreg.predict(testX)

# calculate accuracy
acc = metrics.accuracy_score(testY, predY)
print("test accuracy=", acc)
```

```
w= [[-4.13160888  1.30538269]
```

```
 [-0.71650483  0.23668666]
 [ 4.84811371 -1.54206935]]
b= [ 11.46619303   5.40363007 -16.8698231 ]
test accuracy= 0.9733333333333334
```

```
/Users/jpm/miniconda3/envs/mmtorch/lib/python3.13/site-
packages/sklearn/linear_model/_logistic.py:1272: FutureWarning: 'multi_class'
was deprecated in version 1.5 and will be removed in 1.8. From then on, it will
always use 'multinomial'. Leave it to its default value to avoid this warning.
  warnings.warn(
```

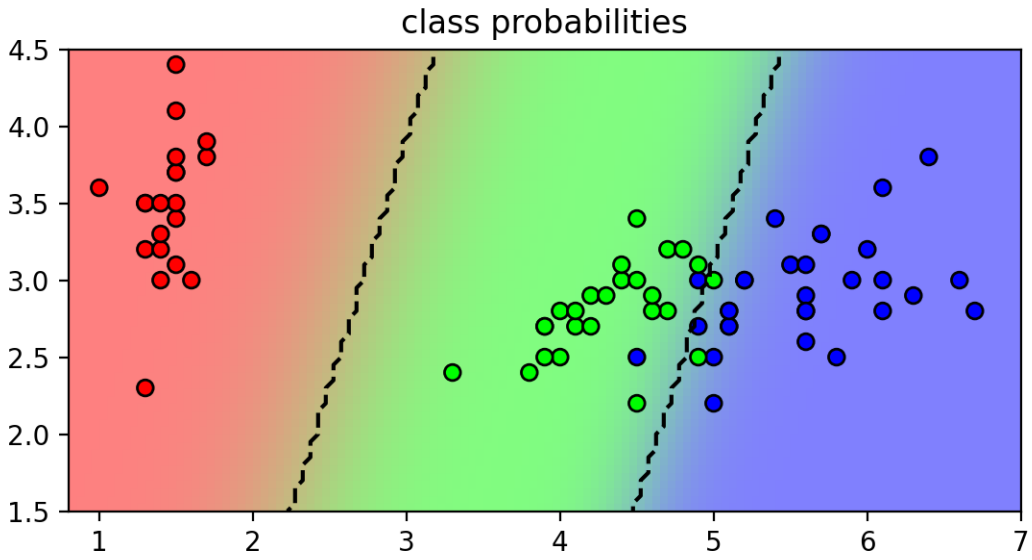
[19]:
```
axbox3 = [0.8, 7, 1.5, 4.5]
# make a colormap for viewing 3 classes
mycmap3 = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#00FF00", "#0000FF"])


def plot_posterior3(model, axbox, mycmap):
    xr = [ arange(0.8,7,0.05) , arange(1.5, 4.5, 0.05) ]

    # make a grid for calculating the posterior,
    #  then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]

    # predict probabilities
    Z = model.predict_proba(allpts)
    P = model.predict(allpts)

    # use probabilities as RGB color
    ZZ = Z.reshape((len(xr[1]), len(xr[0]), 3))

    plt.imshow(ZZ, origin='lower', extent=axbox, alpha=0.50)
    plt.contour(xr[0], xr[1], P.reshape(xgrid0.shape), levels=[0.5,1.5,2.5],␣
 ↪linestyles='dashed', colors='black')
#    irisaxis(axbox)
```

[20]:
```
lr3classm = plt.figure()
plot_posterior3(mlogreg, axbox3, mycmap3)
plt.scatter(trainX[:,0], trainX[:,1], c=trainY, cmap=mycmap3, edgecolors='k')
plt.title('class probabilities');
plt.close()
```

- class probabilites from softmax function.

[21]:
```
lr3classm
```

[21]:



27 Summary

- Two related linear classification models: Perceptron, Multi-class logistic regression
  - compute a linear function of input
  - non-linear output function (threshold or soft-max)
- We have assumed the inputs are feature vectors already.
  - What if we want to extract the features vectors too?

Lecture5b

September 29, 2025

1 CS5489 - Machine Learning

2 Lecture 5b - Neural Networks

2.0.1 Dept. of Computer Science, City University of Hong Kong

[1]:
```
# setup
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100  # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats

rbow = plt.get_cmap('rainbow')
```

3 Outline

- History
- Perceptron
- Multi-class logistic regression
- **Multi-layer perceptron (MLP)**

4 Extracting features

- The multi-class logistic regression model assumes the inputs are feature vectors.
  - $\mathbf{g} = \mathbf{W}^T\mathbf{x}$
  - $\mathbf{f} = \mathbf{s}(\mathbf{g})$
- What if we also want to learn the feature vectors?
  - Replace $\mathbf{x}$ with a feature extractor.
  - For simplicity, we can reuse the same "unit" as the classifier to compute the "features".
- Replace $\mathbf{x}$ with feature extractor $\mathbf{z} = \sigma(\mathbf{A}^T\mathbf{x})$
  - $\mathbf{z}$ is the extracted feature vector (also called *hidden* nodes)

        * $z_i = \sigma(\mathbf{a}_i^T \mathbf{x})$ is a hidden node.
    - $\sigma()$ is the sigmoid function (also called *activation* function.)
    - $\mathbf{A}$ are the parameters of the feature extractor.

- New model
    - $\mathbf{z} = \sigma(\mathbf{A}^T \mathbf{x})$
    - $\mathbf{g} = \mathbf{W}^T \mathbf{z}$
    - $\mathbf{f} = \mathbf{s}(\mathbf{g})$

- **Interpretation**
    - Each hidden node z_i is a "classifier" looking for pattern based on $\mathbf{a}_i$.
    - The output is looking for patterns in the vector $\mathbf{x}$
    - the effect is a non-linear classifier in the input space of $\mathbf{x}$.
    - we can apply this recursively to create features of features ...

5 Multi-layer Perceptron

- Add hidden layers between the inputs and outputs
    - each hidden node is a Perceptron (with its own set of weights)
        * its inputs are the outputs from previous layer
        * extracts a feature pattern from the previous layer
    - can model more complex functions

- Formally, for one layer:
    - $\mathbf{h} = f(\mathbf{W}^T \mathbf{x})$
        * Weight matrix $\mathbf{W}$ - one column for each node
        * Input $\mathbf{x}$ - from previous layer
        * Output $\mathbf{h}$ - to next layer
        * $f(a)$ is the activation function - applied to each dimension to get output
- Also called *fully-connected layers* or *dense layers*

6 Activation functions

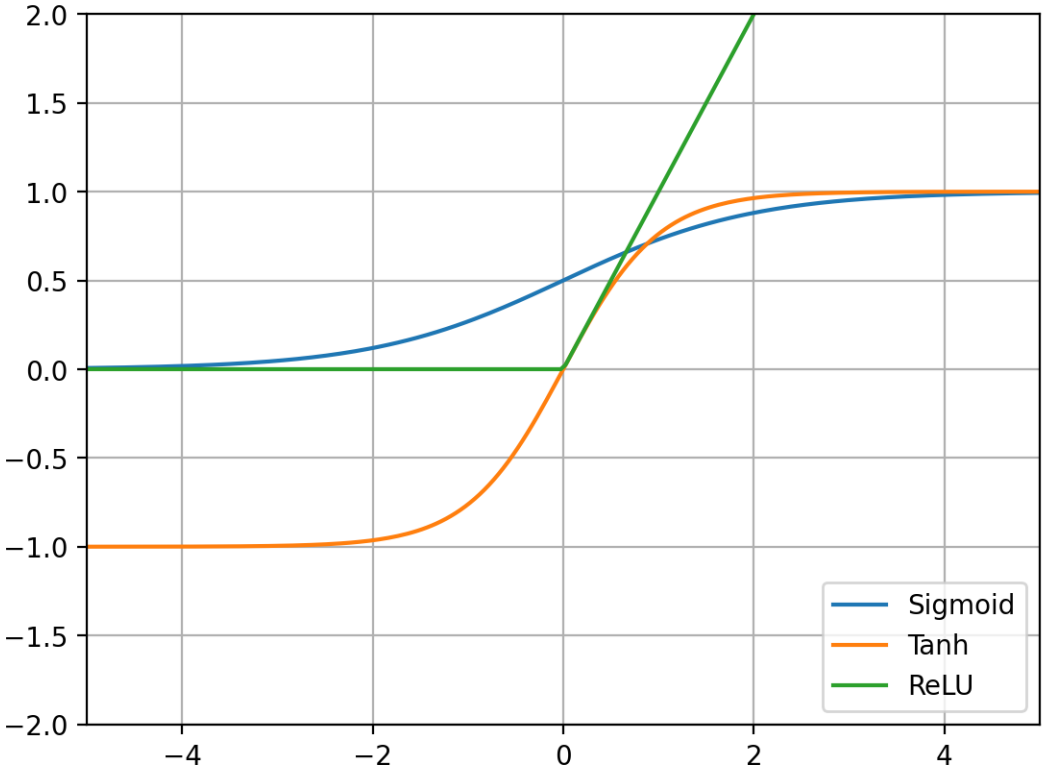
- There are different types of activation functions:
    - *Sigmoid* - output [0,1]
    - *Tanh* - output [-1,1]
    - *Rectifier Linear Unit (ReLU)* - output [0,∞]

[2]:
```
x=linspace(-5,5,200)
fs = {'Sigmoid': lambda x: 1/(1+exp(-x)),
      'Tanh':    lambda x: tanh(x),
      'ReLU':    lambda x: maximum(0,x),
     }

actfig = plt.figure()
for name,f in fs.items():
    plt.plot(x, f(x), label=name)
plt.legend(loc="lower right")
```

```
plt.axis([-5,5,-2,2])
plt.grid(True)
plt.close()
```

[3]:
```
actfig
```

[3]:



- Activation functions specifically for output nodes:
  - *Linear* - output for regression
  - *Softmax* - output for classification (same as multi-class logistic regression)
- Each layer can use a different activation function.

7 Which activation function is best?

- In the early days, only the Sigmoid and Tanh activation functions were used.
  - these were notoriously hard to train with.
    - gradient decays to zero on both ends
- Recently, ReLU has become very popular.
  - easier to train with - gradient is either 0 or 1.
  - faster - don't need to calculate exponential
  - sparse representation - most nodes will output zero.

8 Training an MLP

- Assume a general case:
  - linear transform: $\mathbf{g}_1 = \mathbf{A}^T\mathbf{x}$
  - activation: $\mathbf{z} = h_1(\mathbf{g}_1)$
  - linear transform: $\mathbf{g}_2 = \mathbf{W}^T\mathbf{z}$
  - activation: $\mathbf{f} = h_2(\mathbf{g}_2)$
  - loss function: $L(\mathbf{y}, \mathbf{f})$
- In our example, h_1 is a sigmoid, h_2 is a softmax.
- Similar to multi-class logistic regression
  - minimize the loss
    - $(\mathbf{A}^*, \mathbf{W}^*) = \text{argmin}_{\mathbf{A},\mathbf{W}} L(\mathbf{y}, \mathbf{f}(\mathbf{x}))$
  - use gradient descent as before
    - need to compute $\frac{dL}{d\mathbf{A}}$ and $\frac{dL}{d\mathbf{W}}$
    - we have done most of the work already…
- Computation graph
- Chain rule:
  - 1) $\frac{dL}{d\mathbf{f}}$
  - 2) $\frac{dL}{d\mathbf{g}_2} = \frac{d\mathbf{f}^T}{d\mathbf{g}_2}\frac{dL}{d\mathbf{f}} \Rightarrow$ 3) $\frac{dL}{d\mathbf{w}_j} = \frac{d\mathbf{g}_2^T}{d\mathbf{w}_j}\frac{dL}{d\mathbf{g}_2}$
  - 4) $\frac{dL}{d\mathbf{z}} = \frac{d\mathbf{g}_2^T}{d\mathbf{z}}\frac{dL}{d\mathbf{g}_2}$
  - 5) $\frac{dL}{d\mathbf{g}_1} = \frac{d\mathbf{z}^T}{d\mathbf{g}_1}\frac{dL}{d\mathbf{z}} \Rightarrow$ 6) $\frac{dL}{d\mathbf{a}_j} = \frac{d\mathbf{g}_1^T}{d\mathbf{a}_j}\frac{dL}{d\mathbf{g}_1}$
- Recursively use the gradient of descendant layers
  - propagate gradient backwards
  - backwards propagation
  - back propagation
  - backpropagation
  - backprop
  - BP

9 Backpropagation (backward propagation)

- Defines a set of recursive relationships
  1) calculate the output of each node from first to last layer
  2) calculate the gradient of each node from last to first layer
- **NOTE:** the gradients multiply in each layer!
  - if two gradients are small (<1), their product will be even smaller. This is the "vanishing gradient" problem.

10 Backpropagation (general form)

- Given a computation graph
  1) apply input $\mathbf{x}$ and forward propagate to compute all the nodes' values and the loss.
  2) Go backwards from end, at node $\mathbf{h}$:
  - compute gradients from child nodes: $\frac{dL}{d\mathbf{h}} = \sum_{g \in \text{child}(\mathbf{h})} \frac{d\mathbf{g}^T}{d\mathbf{h}}\frac{dL}{d\mathbf{g}}$

  - compute gradient of parameters $\mathbf{w}_h$ of h: $\frac{dL}{d\mathbf{w}_h} = \frac{d\mathbf{h}^T}{d\mathbf{w}_h}\frac{dL}{d\mathbf{h}}$
- This is the "symbol-to-number" differentiation (e.g., Caffe, Torch).
- We can also make the backprop operations as part of the computation graph.
  - called "symbol-to-symbol" differentiation (e.g., Tensorflow, Theano)

11 Stochastic Gradient Descent (SGD)

- The datasets needed to train NN are typically very large
- Use SGD so that only a small portion of the dataset is needed at a time
  - Each small portion is called a *mini-batch*
  - Use a *momentum* term, which averages the current gradient with those from previous mini-batches.
  - One complete pass through the data is called an *epoch*.

12 Monitoring training with SGD

- Separate the training set into training and validation
  - use the training set to run backpropagation
  - test the NN on the validation set for diagnostics
    * check for convergence - adjust learning rate if necessary
    * check for diverging loss - adjust learning rate
    * stopping criteria - stop when no change in the validation error.
    * decay learning rate after each epoch.

13 Load NN software

- We will use *pytorch*
  - compatible with scikit-learn
  - pytorch is an easy-to-use and widely-utilized framework

[4]:
```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
```

[5]:
```
import sys
print("Python:", sys.version, "Torch", torch.__version__)
from typing import List
```

```
Python: 3.13.7 | packaged by Anaconda, Inc. | (main, Sep  9 2025, 19:54:17)
[Clang 17.0.6 ] Torch 2.8.0
```

[6]:
```
# initialize random seed
torch.manual_seed(4487)
np.random.seed(4487)
```

```
random.seed(4487)
```

[7]:
```
# type three - moons
X0,Y = datasets.make_moons(n_samples=100, noise=0.1, random_state=4487)
mycmap = matplotlib.colors.LinearSegmentedColormap.from_list('mycmap',␣
 ↪["#FF0000", "#FFFFFF", "#00FF00"])

scaler = preprocessing.MinMaxScaler(feature_range=(-1,1))
X = scaler.fit_transform(X0)

axbox = [-1.2, 1.2, -1.2, 1.2]
```

[8]:
```
def plot_nn(model, axbox, X, Y, mode="prob"):
    # assumes a sigmoid output node
    # grid points
    xr = [ linspace(axbox[0], axbox[1], 200),
           linspace(axbox[2], axbox[3], 200) ]

    # make a grid for calculating the posterior,
    #  then form into a big [N,2] matrix
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = torch.as_tensor(c_[xgrid0.ravel(), xgrid1.ravel()], dtype=torch.
 ↪float32)

    # print(xgrid0.shape, xgrid1.shape, allpts.shape)

    # calculate the decision function
    if mode == "prob":
        #scoreall = model.predict_proba(allpts) # tf1
        # score = scoreall[:,1].reshape(xgrid0.shape)
        scoreall = model(allpts).softmax(-1)
        score = scoreall[...,1].view(xgrid0.shape).detach().numpy()
        maxscore = 1.0
        minscore = 0.0
        al = 0.3

    plt.imshow(score, origin='lower', extent=axbox, alpha=al, cmap=mycmap,
               vmin=minscore, vmax=maxscore, aspect='auto')

    plt.contour(xr[0], xr[1], score, levels=[0.5], linestyles='solid',␣
 ↪colors='k',
                linewidths=2)

    plt.scatter(X[:,0], X[:,1], c=Y, cmap=mycmap, edgecolors='k')
    plt.axis(axbox); plt.grid(True)
```

[9]:
```
def plot_history(history):
    plt.plot(history['train_loss'], label="training loss ({:.6f})".
 ↪format(history['train_loss'][-1]))
    plt.plot(history['val_loss'], label="validation loss ({:.6f})".
 ↪format(history['val_loss'][-1]))
    plt.grid(True)
    plt.legend(loc="best", fontsize=9)
    plt.axis([0,len(history['train_loss']), 0, 0.7])
    plt.xlabel('iteration')
    plt.ylabel('loss')
```

[10]:
```
# split training and validation
val_num = int(len(X) * 0.1)  # 10% as validation
arr_permute = np.random.permutation(len(X))
X, Y = X[arr_permute], Y[arr_permute]
trainX, trainY = X[:-val_num].astype(np.float32), Y[:-val_num]
valX, valY = X[-val_num:].astype(np.float32), Y[-val_num:]
```

Let's define a NN module and a training loop with pytorch!

[11]:
```
# define the network
class MyModel(nn.Module):
    def __init__(self, input_size: int, output_size: int, hidden_sizes:␣
 ↪List[int]):
        super().__init__()
        self.input_size, self.output_size, self.hidden_sizes = input_size,␣
 ↪output_size, hidden_sizes
        layers = []
        # for the hidden layers
        prev_size = input_size
        for curr_size in hidden_sizes:
            layers.append(nn.Linear(prev_size, curr_size))  # [bs, hidden_size]
            layers.append(nn.ReLU()) # we use relu activation
            prev_size = curr_size
        # for the output layer
        layers.append(nn.Linear(prev_size, output_size))  # [bs, output_size]
        # layers.append(nn.Softmax(dim=-1))  # we perform softmax at the␣
 ↪outside, not here!
        # add to the Module itself
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        t_input = x.view(-1, self.input_size)  # [bs, input_size]
        t_output = self.network(t_input)  # [bs, output_size]
        return t_output
```

[12]:
```python
# A summary function to see the parameters of a module
def summary_nn(mod):
    total_numel = 0
    print(mod)
    for name, param in mod.named_parameters():
        total_numel += param.numel()
        print(f"Param: {name}\tShape: {list(param.shape)}\tNumel: {param.
 ↪numel()}")
    print(f"Total Numel = {total_numel}")
```

14 Overfitting

- Continuous training will sometimes lead to overfitting
  - the training loss decreases, but the validation loss increases

15 Early stopping

- Training can be stopped when the validation loss is stable for a number of iterations
  - stable means change below a threshold
  - this is to prevent overfitting the training data.
  - we can limit the number of iterations.
- *We are using the loss/accuracy on the (held-out) validation data to estimate the generalization performance of the network*

[13]:
```python
# Early stopping implementation
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.0001):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_accuracy = 0.0
        self.early_stop = False

    def __call__(self, current_accuracy):
        if current_accuracy - self.best_accuracy > self.min_delta:
            self.best_accuracy = current_accuracy
            self.counter = 0
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        return self.early_stop
```

[14]:
```python
# define the training loop
def train_model(model, criterion, train_loader, valid_loader, total_epoch: int,␣
 ↪patience: int):
```

```
    # for early stopping
    early_stopping = EarlyStopping(patience=patience)
    # record all the losses and accuracies
    train_losses = []
    train_accs = []
    val_losses = []
    val_accs = []
    # all the epochs
    for epoch in range(total_epoch):
        model.train()  # set to training mode
        running_loss = 0.0
        correct = 0
        total = 0
        # loop over the training data
        for inputs, labels in train_loader:
            optimizer.zero_grad()  # 1. Zero the gradients
            outputs = model(inputs)  # 2. Forward pass
            loss = criterion(outputs, labels)  # 3. Calculate the loss
            loss.backward()  # 4. Backward pass
            optimizer.step()  # 5. Optimizer step
            # recordings
            with torch.no_grad():
                running_loss += loss.item()
                predicted = torch.argmax(outputs, -1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()
        # results for one epoch
        train_loss = running_loss / len(train_loader)
        train_acc = correct / total
        train_losses.append(train_loss)
        train_accs.append(train_acc)
        # Validation
        model.eval()  # set to eval mode
        val_loss = 0.0
        val_correct = 0
        val_total = 0
        with torch.no_grad():
            for inputs, labels in valid_loader:
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                val_loss += loss.item()
                predicted = torch.argmax(outputs, -1)
                val_total += labels.size(0)
                val_correct += (predicted == labels).sum().item()
        val_loss = val_loss / len(valid_loader)
        val_acc = val_correct / val_total
        val_losses.append(val_loss)
```

```
        val_accs.append(val_acc)
        # print results
        print(f'Epoch {epoch+1}: Train Loss: {train_loss:.4f}, Train Acc:␣
 ↪{train_acc:.4f}, Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}')
        if early_stopping(val_acc):
            print("Early stopping triggered")
            break
    return {'train_loss': train_losses, 'train_acc': train_accs, 'val_loss':␣
 ↪val_losses, 'val_acc': val_accs}
```

[15]:
```
# get dataset and dataloader
def get_dataloader(t_input, t_output, batch_size=32, shuffle=False):
    t_input, t_output = torch.as_tensor(t_input), torch.as_tensor(t_output)
    dataset = TensorDataset(t_input, t_output)
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=shuffle)
    return loader
```

- train 1 NN with just one output layer
  - this is the same as logistic regression

[16]:
```
model = MyModel(input_size=2, output_size=2, hidden_sizes=[])  # one layer model
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True) ␣
 ↪# optimizer
train_loader = get_dataloader(trainX, trainY)  # data loaders
val_loader = get_dataloader(valX, valY)
# train the model
history = train_model(model, criterion, train_loader, val_loader,␣
 ↪total_epoch=100, patience=50)
```

```
Epoch 1: Train Loss: 1.0289, Train Acc: 0.2556, Val Loss: 0.8489, Val Acc:
0.2000
Epoch 2: Train Loss: 0.7875, Train Acc: 0.4000, Val Loss: 0.6500, Val Acc:
0.7000
Epoch 3: Train Loss: 0.5473, Train Acc: 0.7889, Val Loss: 0.5168, Val Acc:
0.8000
Epoch 4: Train Loss: 0.3962, Train Acc: 0.8778, Val Loss: 0.4531, Val Acc:
0.8000
Epoch 5: Train Loss: 0.3205, Train Acc: 0.8889, Val Loss: 0.4296, Val Acc:
0.8000
Epoch 6: Train Loss: 0.2851, Train Acc: 0.8889, Val Loss: 0.4257, Val Acc:
0.8000
Epoch 7: Train Loss: 0.2685, Train Acc: 0.8889, Val Loss: 0.4302, Val Acc:
0.8000
Epoch 8: Train Loss: 0.2607, Train Acc: 0.8889, Val Loss: 0.4375, Val Acc:
0.8000
Epoch 9: Train Loss: 0.2569, Train Acc: 0.8889, Val Loss: 0.4448, Val Acc:
0.8000
```

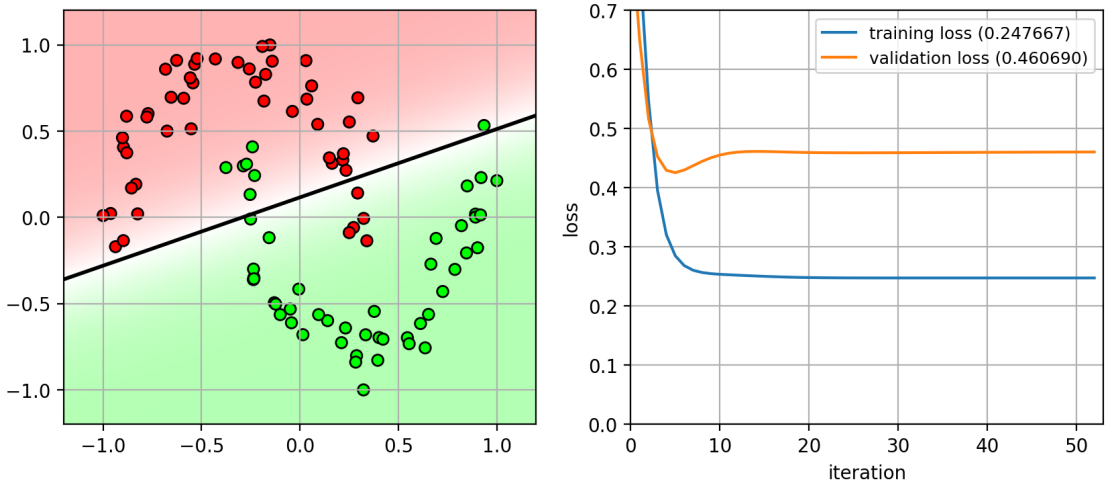
```
Epoch 10: Train Loss: 0.2549, Train Acc: 0.8889, Val Loss: 0.4509, Val Acc:
0.8000
Epoch 11: Train Loss: 0.2538, Train Acc: 0.8889, Val Loss: 0.4555, Val Acc:
0.8000
Epoch 12: Train Loss: 0.2530, Train Acc: 0.8889, Val Loss: 0.4586, Val Acc:
0.8000
Epoch 13: Train Loss: 0.2523, Train Acc: 0.8889, Val Loss: 0.4604, Val Acc:
0.8000
Epoch 14: Train Loss: 0.2517, Train Acc: 0.8889, Val Loss: 0.4613, Val Acc:
0.8000
Epoch 15: Train Loss: 0.2510, Train Acc: 0.8889, Val Loss: 0.4616, Val Acc:
0.8000
Epoch 16: Train Loss: 0.2504, Train Acc: 0.8889, Val Loss: 0.4615, Val Acc:
0.8000
Epoch 17: Train Loss: 0.2498, Train Acc: 0.8889, Val Loss: 0.4611, Val Acc:
0.8000
Epoch 18: Train Loss: 0.2493, Train Acc: 0.8889, Val Loss: 0.4608, Val Acc:
0.8000
Epoch 19: Train Loss: 0.2489, Train Acc: 0.9000, Val Loss: 0.4604, Val Acc:
0.8000
Epoch 20: Train Loss: 0.2486, Train Acc: 0.9000, Val Loss: 0.4601, Val Acc:
0.8000
Epoch 21: Train Loss: 0.2483, Train Acc: 0.9000, Val Loss: 0.4598, Val Acc:
0.8000
Epoch 22: Train Loss: 0.2481, Train Acc: 0.8889, Val Loss: 0.4596, Val Acc:
0.8000
Epoch 23: Train Loss: 0.2480, Train Acc: 0.8889, Val Loss: 0.4595, Val Acc:
0.8000
Epoch 24: Train Loss: 0.2479, Train Acc: 0.8889, Val Loss: 0.4594, Val Acc:
0.8000
Epoch 25: Train Loss: 0.2478, Train Acc: 0.8889, Val Loss: 0.4593, Val Acc:
0.8000
Epoch 26: Train Loss: 0.2478, Train Acc: 0.8889, Val Loss: 0.4593, Val Acc:
0.8000
Epoch 27: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4593, Val Acc:
0.8000
Epoch 28: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4593, Val Acc:
0.8000
Epoch 29: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4593, Val Acc:
0.8000
Epoch 30: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4594, Val Acc:
0.8000
Epoch 31: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4594, Val Acc:
0.8000
Epoch 32: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4595, Val Acc:
0.8000
Epoch 33: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4596, Val Acc:
0.8000
```

```
Epoch 34: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4596, Val Acc:
0.8000
Epoch 35: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4597, Val Acc:
0.8000
Epoch 36: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4598, Val Acc:
0.8000
Epoch 37: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4599, Val Acc:
0.8000
Epoch 38: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4599, Val Acc:
0.8000
Epoch 39: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4600, Val Acc:
0.8000
Epoch 40: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4601, Val Acc:
0.8000
Epoch 41: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4601, Val Acc:
0.8000
Epoch 42: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4602, Val Acc:
0.8000
Epoch 43: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4602, Val Acc:
0.8000
Epoch 44: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4603, Val Acc:
0.8000
Epoch 45: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4604, Val Acc:
0.8000
Epoch 46: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4604, Val Acc:
0.8000
Epoch 47: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4605, Val Acc:
0.8000
Epoch 48: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4605, Val Acc:
0.8000
Epoch 49: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4605, Val Acc:
0.8000
Epoch 50: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4606, Val Acc:
0.8000
Epoch 51: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4606, Val Acc:
0.8000
Epoch 52: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4607, Val Acc:
0.8000
Epoch 53: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4607, Val Acc:
0.8000
Early stopping triggered
```

[17]:
```
nnfig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plot_nn(model, axbox, X, Y)
plt.subplot(1,2,2)
plot_history(history)
```

```
plt.close()

nnfig
```

[17]:



- Add one 1 hidden layer with 4 ReLU nodes

[18]:

```
model = MyModel(input_size=2, output_size=2, hidden_sizes=[4])  # MLP model
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.3, momentum=0.9, nesterov=True)
 ↪# optimizer
train_loader = get_dataloader(trainX, trainY)  # data loaders
val_loader = get_dataloader(valX, valY)
# train the model
history = train_model(model, criterion, train_loader, val_loader,
 ↪total_epoch=100, patience=50)
```

```
Epoch 1: Train Loss: 0.6305, Train Acc: 0.5889, Val Loss: 0.5638, Val Acc:
0.7000
Epoch 2: Train Loss: 0.4554, Train Acc: 0.8444, Val Loss: 0.4498, Val Acc:
0.7000
Epoch 3: Train Loss: 0.3019, Train Acc: 0.8667, Val Loss: 0.4678, Val Acc:
0.8000
Epoch 4: Train Loss: 0.2763, Train Acc: 0.8889, Val Loss: 0.5332, Val Acc:
0.8000
Epoch 5: Train Loss: 0.2843, Train Acc: 0.9000, Val Loss: 0.5601, Val Acc:
0.8000
Epoch 6: Train Loss: 0.2938, Train Acc: 0.8778, Val Loss: 0.5586, Val Acc:
0.8000
Epoch 7: Train Loss: 0.2960, Train Acc: 0.8667, Val Loss: 0.5324, Val Acc:
0.8000
Epoch 8: Train Loss: 0.2872, Train Acc: 0.8667, Val Loss: 0.4952, Val Acc:
```

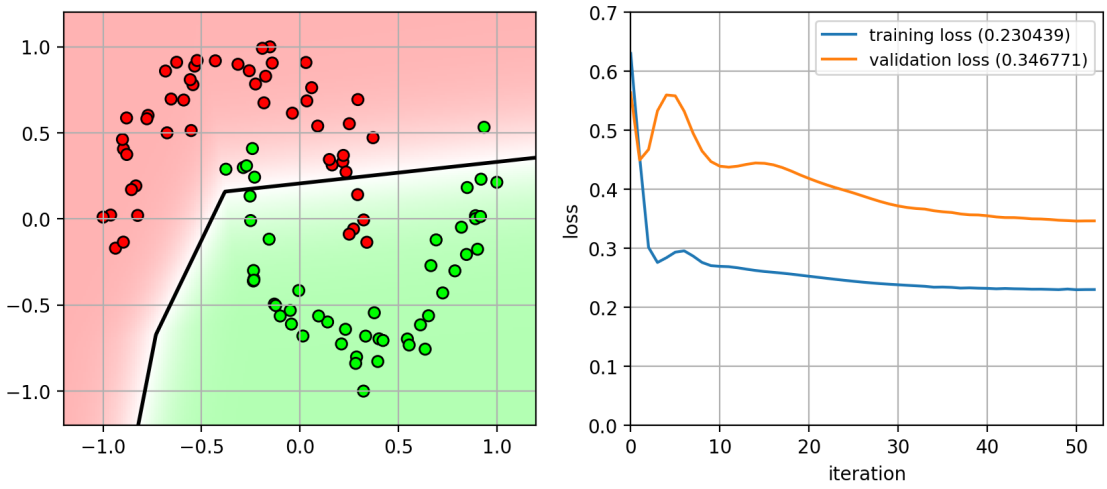
```
0.8000
Epoch 9: Train Loss: 0.2764, Train Acc: 0.8667, Val Loss: 0.4652, Val Acc:
0.8000
Epoch 10: Train Loss: 0.2709, Train Acc: 0.9000, Val Loss: 0.4473, Val Acc:
0.8000
Epoch 11: Train Loss: 0.2698, Train Acc: 0.9000, Val Loss: 0.4394, Val Acc:
0.8000
Epoch 12: Train Loss: 0.2691, Train Acc: 0.9000, Val Loss: 0.4378, Val Acc:
0.8000
Epoch 13: Train Loss: 0.2673, Train Acc: 0.9000, Val Loss: 0.4395, Val Acc:
0.8000
Epoch 14: Train Loss: 0.2649, Train Acc: 0.9000, Val Loss: 0.4425, Val Acc:
0.8000
Epoch 15: Train Loss: 0.2627, Train Acc: 0.9000, Val Loss: 0.4449, Val Acc:
0.8000
Epoch 16: Train Loss: 0.2610, Train Acc: 0.9000, Val Loss: 0.4443, Val Acc:
0.8000
Epoch 17: Train Loss: 0.2596, Train Acc: 0.9000, Val Loss: 0.4417, Val Acc:
0.8000
Epoch 18: Train Loss: 0.2582, Train Acc: 0.8889, Val Loss: 0.4372, Val Acc:
0.8000
Epoch 19: Train Loss: 0.2566, Train Acc: 0.8889, Val Loss: 0.4312, Val Acc:
0.8000
Epoch 20: Train Loss: 0.2548, Train Acc: 0.8889, Val Loss: 0.4247, Val Acc:
0.8000
Epoch 21: Train Loss: 0.2530, Train Acc: 0.8889, Val Loss: 0.4186, Val Acc:
0.8000
Epoch 22: Train Loss: 0.2512, Train Acc: 0.8889, Val Loss: 0.4129, Val Acc:
0.8000
Epoch 23: Train Loss: 0.2494, Train Acc: 0.8889, Val Loss: 0.4077, Val Acc:
0.8000
Epoch 24: Train Loss: 0.2477, Train Acc: 0.8889, Val Loss: 0.4029, Val Acc:
0.8000
Epoch 25: Train Loss: 0.2460, Train Acc: 0.8889, Val Loss: 0.3985, Val Acc:
0.8000
Epoch 26: Train Loss: 0.2444, Train Acc: 0.8889, Val Loss: 0.3940, Val Acc:
0.8000
Epoch 27: Train Loss: 0.2429, Train Acc: 0.8889, Val Loss: 0.3891, Val Acc:
0.8000
Epoch 28: Train Loss: 0.2416, Train Acc: 0.8889, Val Loss: 0.3842, Val Acc:
0.8000
Epoch 29: Train Loss: 0.2404, Train Acc: 0.8889, Val Loss: 0.3795, Val Acc:
0.8000
Epoch 30: Train Loss: 0.2394, Train Acc: 0.8889, Val Loss: 0.3754, Val Acc:
0.8000
Epoch 31: Train Loss: 0.2385, Train Acc: 0.8889, Val Loss: 0.3721, Val Acc:
0.8000
Epoch 32: Train Loss: 0.2376, Train Acc: 0.8889, Val Loss: 0.3695, Val Acc:
```

```
0.8000
Epoch 33: Train Loss: 0.2368, Train Acc: 0.8889, Val Loss: 0.3677, Val Acc:
0.8000
Epoch 34: Train Loss: 0.2359, Train Acc: 0.9000, Val Loss: 0.3668, Val Acc:
0.8000
Epoch 35: Train Loss: 0.2344, Train Acc: 0.9000, Val Loss: 0.3640, Val Acc:
0.8000
Epoch 36: Train Loss: 0.2346, Train Acc: 0.9000, Val Loss: 0.3622, Val Acc:
0.8000
Epoch 37: Train Loss: 0.2341, Train Acc: 0.9000, Val Loss: 0.3610, Val Acc:
0.8000
Epoch 38: Train Loss: 0.2329, Train Acc: 0.9000, Val Loss: 0.3585, Val Acc:
0.8000
Epoch 39: Train Loss: 0.2333, Train Acc: 0.9000, Val Loss: 0.3571, Val Acc:
0.8000
Epoch 40: Train Loss: 0.2327, Train Acc: 0.9000, Val Loss: 0.3568, Val Acc:
0.8000
Epoch 41: Train Loss: 0.2324, Train Acc: 0.9000, Val Loss: 0.3554, Val Acc:
0.8000
Epoch 42: Train Loss: 0.2317, Train Acc: 0.9000, Val Loss: 0.3535, Val Acc:
0.8000
Epoch 43: Train Loss: 0.2322, Train Acc: 0.9000, Val Loss: 0.3524, Val Acc:
0.8000
Epoch 44: Train Loss: 0.2317, Train Acc: 0.9000, Val Loss: 0.3523, Val Acc:
0.8000
Epoch 45: Train Loss: 0.2315, Train Acc: 0.9000, Val Loss: 0.3515, Val Acc:
0.8000
Epoch 46: Train Loss: 0.2310, Train Acc: 0.9000, Val Loss: 0.3502, Val Acc:
0.8000
Epoch 47: Train Loss: 0.2311, Train Acc: 0.9000, Val Loss: 0.3500, Val Acc:
0.8000
Epoch 48: Train Loss: 0.2307, Train Acc: 0.9000, Val Loss: 0.3491, Val Acc:
0.8000
Epoch 49: Train Loss: 0.2303, Train Acc: 0.9000, Val Loss: 0.3479, Val Acc:
0.8000
Epoch 50: Train Loss: 0.2313, Train Acc: 0.9000, Val Loss: 0.3473, Val Acc:
0.8000
Epoch 51: Train Loss: 0.2301, Train Acc: 0.9000, Val Loss: 0.3465, Val Acc:
0.8000
Epoch 52: Train Loss: 0.2304, Train Acc: 0.9000, Val Loss: 0.3467, Val Acc:
0.8000
Epoch 53: Train Loss: 0.2304, Train Acc: 0.9000, Val Loss: 0.3468, Val Acc:
0.8000
Early stopping triggered
```

[19]:
```
nnfig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
```

```
plot_nn(model, axbox, X, Y)
plt.subplot(1,2,2)
plot_history(history)
plt.close()

nnfig
```

[19]:



- Let's try more nodes
  - 1 hidden layer with 20 hidden nodes

[20]:
```
model = MyModel(input_size=2, output_size=2, hidden_sizes=[20])  # MLP model
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.3, momentum=0.9, nesterov=True)
 ↪# optimizer
train_loader = get_dataloader(trainX, trainY)  # data loaders
val_loader = get_dataloader(valX, valY)
# train the model
history = train_model(model, criterion, train_loader, val_loader,
 ↪total_epoch=100, patience=50)
```

```
Epoch 1: Train Loss: 0.6014, Train Acc: 0.6778, Val Loss: 0.4945, Val Acc:
0.7000
Epoch 2: Train Loss: 0.3375, Train Acc: 0.8889, Val Loss: 0.4680, Val Acc:
0.8000
Epoch 3: Train Loss: 0.2777, Train Acc: 0.8889, Val Loss: 0.5161, Val Acc:
0.8000
Epoch 4: Train Loss: 0.2792, Train Acc: 0.9000, Val Loss: 0.5402, Val Acc:
0.8000
Epoch 5: Train Loss: 0.2866, Train Acc: 0.8778, Val Loss: 0.5418, Val Acc:
0.8000
Epoch 6: Train Loss: 0.2877, Train Acc: 0.8667, Val Loss: 0.5130, Val Acc:
```

```
0.8000
Epoch 7: Train Loss: 0.2744, Train Acc: 0.8778, Val Loss: 0.4772, Val Acc:
0.8000
Epoch 8: Train Loss: 0.2636, Train Acc: 0.8778, Val Loss: 0.4544, Val Acc:
0.8000
Epoch 9: Train Loss: 0.2595, Train Acc: 0.8889, Val Loss: 0.4366, Val Acc:
0.8000
Epoch 10: Train Loss: 0.2569, Train Acc: 0.8778, Val Loss: 0.4291, Val Acc:
0.8000
Epoch 11: Train Loss: 0.2505, Train Acc: 0.8889, Val Loss: 0.4258, Val Acc:
0.8000
Epoch 12: Train Loss: 0.2425, Train Acc: 0.8889, Val Loss: 0.4277, Val Acc:
0.8000
Epoch 13: Train Loss: 0.2352, Train Acc: 0.8778, Val Loss: 0.4295, Val Acc:
0.8000
Epoch 14: Train Loss: 0.2299, Train Acc: 0.8778, Val Loss: 0.4142, Val Acc:
0.8000
Epoch 15: Train Loss: 0.2241, Train Acc: 0.8778, Val Loss: 0.3980, Val Acc:
0.8000
Epoch 16: Train Loss: 0.2162, Train Acc: 0.8889, Val Loss: 0.3850, Val Acc:
0.8000
Epoch 17: Train Loss: 0.2090, Train Acc: 0.9000, Val Loss: 0.3621, Val Acc:
0.8000
Epoch 18: Train Loss: 0.1997, Train Acc: 0.9000, Val Loss: 0.3441, Val Acc:
0.8000
Epoch 19: Train Loss: 0.1893, Train Acc: 0.9000, Val Loss: 0.3268, Val Acc:
0.8000
Epoch 20: Train Loss: 0.1789, Train Acc: 0.9111, Val Loss: 0.3062, Val Acc:
0.8000
Epoch 21: Train Loss: 0.1670, Train Acc: 0.9222, Val Loss: 0.2840, Val Acc:
0.8000
Epoch 22: Train Loss: 0.1548, Train Acc: 0.9222, Val Loss: 0.2639, Val Acc:
0.8000
Epoch 23: Train Loss: 0.1425, Train Acc: 0.9222, Val Loss: 0.2403, Val Acc:
0.9000
Epoch 24: Train Loss: 0.1294, Train Acc: 0.9222, Val Loss: 0.2190, Val Acc:
0.9000
Epoch 25: Train Loss: 0.1167, Train Acc: 0.9333, Val Loss: 0.1999, Val Acc:
0.9000
Epoch 26: Train Loss: 0.1048, Train Acc: 0.9444, Val Loss: 0.1847, Val Acc:
0.9000
Epoch 27: Train Loss: 0.0937, Train Acc: 0.9444, Val Loss: 0.1695, Val Acc:
0.9000
Epoch 28: Train Loss: 0.0833, Train Acc: 0.9889, Val Loss: 0.1537, Val Acc:
0.9000
Epoch 29: Train Loss: 0.0739, Train Acc: 0.9889, Val Loss: 0.1400, Val Acc:
0.9000
Epoch 30: Train Loss: 0.0655, Train Acc: 0.9889, Val Loss: 0.1291, Val Acc:
```

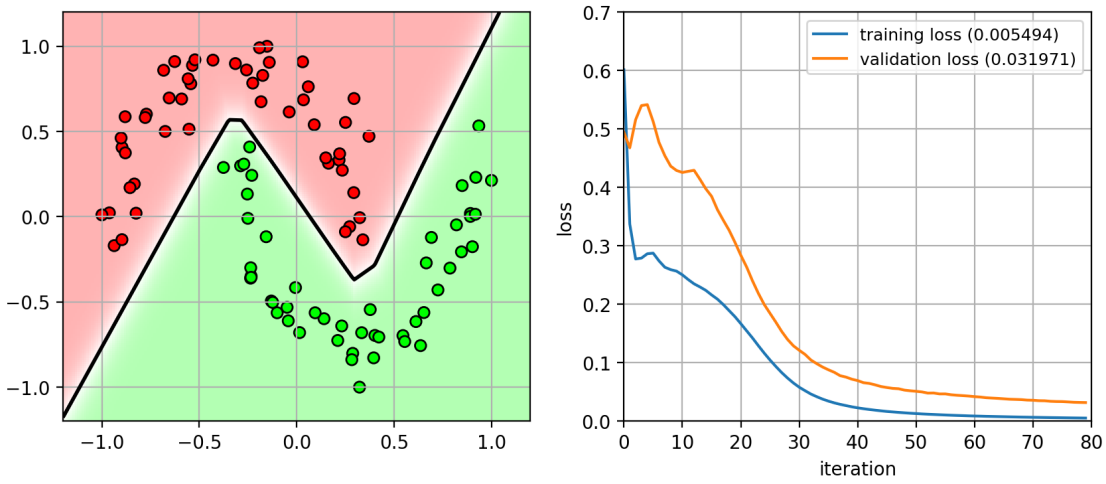
```
1.0000
Epoch 31: Train Loss: 0.0582, Train Acc: 1.0000, Val Loss: 0.1214, Val Acc:
1.0000
Epoch 32: Train Loss: 0.0522, Train Acc: 1.0000, Val Loss: 0.1143, Val Acc:
1.0000
Epoch 33: Train Loss: 0.0465, Train Acc: 1.0000, Val Loss: 0.1048, Val Acc:
1.0000
Epoch 34: Train Loss: 0.0420, Train Acc: 1.0000, Val Loss: 0.0985, Val Acc:
1.0000
Epoch 35: Train Loss: 0.0379, Train Acc: 1.0000, Val Loss: 0.0928, Val Acc:
1.0000
Epoch 36: Train Loss: 0.0344, Train Acc: 1.0000, Val Loss: 0.0880, Val Acc:
1.0000
Epoch 37: Train Loss: 0.0314, Train Acc: 1.0000, Val Loss: 0.0837, Val Acc:
1.0000
Epoch 38: Train Loss: 0.0289, Train Acc: 1.0000, Val Loss: 0.0780, Val Acc:
1.0000
Epoch 39: Train Loss: 0.0268, Train Acc: 1.0000, Val Loss: 0.0756, Val Acc:
1.0000
Epoch 40: Train Loss: 0.0247, Train Acc: 1.0000, Val Loss: 0.0720, Val Acc:
1.0000
Epoch 41: Train Loss: 0.0230, Train Acc: 1.0000, Val Loss: 0.0696, Val Acc:
1.0000
Epoch 42: Train Loss: 0.0214, Train Acc: 1.0000, Val Loss: 0.0660, Val Acc:
1.0000
Epoch 43: Train Loss: 0.0201, Train Acc: 1.0000, Val Loss: 0.0647, Val Acc:
1.0000
Epoch 44: Train Loss: 0.0189, Train Acc: 1.0000, Val Loss: 0.0622, Val Acc:
1.0000
Epoch 45: Train Loss: 0.0178, Train Acc: 1.0000, Val Loss: 0.0594, Val Acc:
1.0000
Epoch 46: Train Loss: 0.0168, Train Acc: 1.0000, Val Loss: 0.0577, Val Acc:
1.0000
Epoch 47: Train Loss: 0.0160, Train Acc: 1.0000, Val Loss: 0.0563, Val Acc:
1.0000
Epoch 48: Train Loss: 0.0151, Train Acc: 1.0000, Val Loss: 0.0556, Val Acc:
1.0000
Epoch 49: Train Loss: 0.0144, Train Acc: 1.0000, Val Loss: 0.0533, Val Acc:
1.0000
Epoch 50: Train Loss: 0.0137, Train Acc: 1.0000, Val Loss: 0.0521, Val Acc:
1.0000
Epoch 51: Train Loss: 0.0132, Train Acc: 1.0000, Val Loss: 0.0513, Val Acc:
1.0000
Epoch 52: Train Loss: 0.0126, Train Acc: 1.0000, Val Loss: 0.0502, Val Acc:
1.0000
Epoch 53: Train Loss: 0.0120, Train Acc: 1.0000, Val Loss: 0.0482, Val Acc:
1.0000
Epoch 54: Train Loss: 0.0116, Train Acc: 1.0000, Val Loss: 0.0484, Val Acc:
```

```
1.0000
Epoch 55: Train Loss: 0.0111, Train Acc: 1.0000, Val Loss: 0.0468, Val Acc:
1.0000
Epoch 56: Train Loss: 0.0107, Train Acc: 1.0000, Val Loss: 0.0468, Val Acc:
1.0000
Epoch 57: Train Loss: 0.0103, Train Acc: 1.0000, Val Loss: 0.0453, Val Acc:
1.0000
Epoch 58: Train Loss: 0.0099, Train Acc: 1.0000, Val Loss: 0.0445, Val Acc:
1.0000
Epoch 59: Train Loss: 0.0096, Train Acc: 1.0000, Val Loss: 0.0438, Val Acc:
1.0000
Epoch 60: Train Loss: 0.0093, Train Acc: 1.0000, Val Loss: 0.0431, Val Acc:
1.0000
Epoch 61: Train Loss: 0.0090, Train Acc: 1.0000, Val Loss: 0.0421, Val Acc:
1.0000
Epoch 62: Train Loss: 0.0087, Train Acc: 1.0000, Val Loss: 0.0414, Val Acc:
1.0000
Epoch 63: Train Loss: 0.0084, Train Acc: 1.0000, Val Loss: 0.0402, Val Acc:
1.0000
Epoch 64: Train Loss: 0.0082, Train Acc: 1.0000, Val Loss: 0.0397, Val Acc:
1.0000
Epoch 65: Train Loss: 0.0080, Train Acc: 1.0000, Val Loss: 0.0391, Val Acc:
1.0000
Epoch 66: Train Loss: 0.0077, Train Acc: 1.0000, Val Loss: 0.0384, Val Acc:
1.0000
Epoch 67: Train Loss: 0.0075, Train Acc: 1.0000, Val Loss: 0.0377, Val Acc:
1.0000
Epoch 68: Train Loss: 0.0073, Train Acc: 1.0000, Val Loss: 0.0374, Val Acc:
1.0000
Epoch 69: Train Loss: 0.0071, Train Acc: 1.0000, Val Loss: 0.0370, Val Acc:
1.0000
Epoch 70: Train Loss: 0.0069, Train Acc: 1.0000, Val Loss: 0.0362, Val Acc:
1.0000
Epoch 71: Train Loss: 0.0068, Train Acc: 1.0000, Val Loss: 0.0359, Val Acc:
1.0000
Epoch 72: Train Loss: 0.0066, Train Acc: 1.0000, Val Loss: 0.0352, Val Acc:
1.0000
Epoch 73: Train Loss: 0.0064, Train Acc: 1.0000, Val Loss: 0.0350, Val Acc:
1.0000
Epoch 74: Train Loss: 0.0063, Train Acc: 1.0000, Val Loss: 0.0342, Val Acc:
1.0000
Epoch 75: Train Loss: 0.0061, Train Acc: 1.0000, Val Loss: 0.0338, Val Acc:
1.0000
Epoch 76: Train Loss: 0.0060, Train Acc: 1.0000, Val Loss: 0.0336, Val Acc:
1.0000
Epoch 77: Train Loss: 0.0059, Train Acc: 1.0000, Val Loss: 0.0331, Val Acc:
1.0000
Epoch 78: Train Loss: 0.0057, Train Acc: 1.0000, Val Loss: 0.0325, Val Acc:
```

```
1.0000
Epoch 79: Train Loss: 0.0056, Train Acc: 1.0000, Val Loss: 0.0322, Val Acc:
1.0000
Epoch 80: Train Loss: 0.0055, Train Acc: 1.0000, Val Loss: 0.0320, Val Acc:
1.0000
Early stopping triggered
```

[21]:
```
nnfig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plot_nn(model, axbox, X, Y)
plt.subplot(1,2,2)
plot_history(history)
plt.close()

nnfig
```

[21]:



16 Universal Approximation Theorem

- Cybenko (1989), Hornik (1991)
  - *A multi-layer perceptron with a single hidden layer and a finite number of nodes can approximate any continuous function up to a desired error.*
    - The number of nodes needed might be very large.
    - Doesn't say anything about how difficult it is to train it.

- *How many hidden nodes are needed?*
  - In the worst case, consider binary functions mapping $\{0,1\}^n \to \{0,1\}$
    - there are 2^n possible inputs.
    - there are 2^{2^n} possible functions (each input has 2 possible outputs)
    - thus, we need 2^n bits in the hidden layer to select one of these functions.
  - need $O(2^n)$ nodes in the hidden layer, exponential in the size of the input!

- *How to train this model?*
  - According to the "no free lunch theorem", there is no universally best learning algorithm!
- Deep learning corollary
  - *The number of functions representable by a deep network requires an exponential number of nodes for a shallow network with 1 hidden-layer.*
  - A deep network can learn the same function using less nodes.
  - Given the same number of nodes, a deep network can learn more complex functions.
- Doesn't say anything about how difficult it is to train it.

17 Example

- Network with 1 hidden layer
  - input (2D) -> 40 hidden nodes -> output (2D)

[22]:
```
trainX2 = vstack((trainX, trainX+array([1.2,0], dtype=np.float32)))
valX2 = vstack((valX, valX+array([1.2,0], dtype=np.float32)))
trainY2 = r_[trainY, trainY]
valY2 = r_[valY, valY]
axbox2 = [-1.2, 2.5, -1.2, 1.2]
```

[23]:
```
model = MyModel(input_size=2, output_size=2, hidden_sizes=[40])  # MLP model
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.2, momentum=0.9, nesterov=True)
 ↪# optimizer
train_loader = get_dataloader(trainX2, trainY2)  # data loaders
val_loader = get_dataloader(valX2, valY2)
# train the model
history = train_model(model, criterion, train_loader, val_loader,
 ↪total_epoch=100, patience=50)
```

```
Epoch 1: Train Loss: 0.5441, Train Acc: 0.6833, Val Loss: 0.4883, Val Acc:
0.8000
Epoch 2: Train Loss: 0.3665, Train Acc: 0.8500, Val Loss: 0.5378, Val Acc:
0.8000
Epoch 3: Train Loss: 0.3475, Train Acc: 0.8500, Val Loss: 0.5654, Val Acc:
0.7000
Epoch 4: Train Loss: 0.3257, Train Acc: 0.8444, Val Loss: 0.4911, Val Acc:
0.8500
Epoch 5: Train Loss: 0.3034, Train Acc: 0.8556, Val Loss: 0.4588, Val Acc:
0.8000
Epoch 6: Train Loss: 0.3019, Train Acc: 0.8611, Val Loss: 0.4507, Val Acc:
0.8000
Epoch 7: Train Loss: 0.2926, Train Acc: 0.8556, Val Loss: 0.4476, Val Acc:
0.8000
Epoch 8: Train Loss: 0.2890, Train Acc: 0.8500, Val Loss: 0.4448, Val Acc:
0.8500
Epoch 9: Train Loss: 0.2840, Train Acc: 0.8500, Val Loss: 0.4432, Val Acc:
```

```
0.8500
Epoch 10: Train Loss: 0.2790, Train Acc: 0.8556, Val Loss: 0.4415, Val Acc:
0.8500
Epoch 11: Train Loss: 0.2750, Train Acc: 0.8667, Val Loss: 0.4326, Val Acc:
0.8000
Epoch 12: Train Loss: 0.2723, Train Acc: 0.8667, Val Loss: 0.4278, Val Acc:
0.8500
Epoch 13: Train Loss: 0.2678, Train Acc: 0.8778, Val Loss: 0.4207, Val Acc:
0.8500
Epoch 14: Train Loss: 0.2659, Train Acc: 0.8722, Val Loss: 0.4108, Val Acc:
0.8500
Epoch 15: Train Loss: 0.2627, Train Acc: 0.8778, Val Loss: 0.4045, Val Acc:
0.8500
Epoch 16: Train Loss: 0.2542, Train Acc: 0.8833, Val Loss: 0.3963, Val Acc:
0.8500
Epoch 17: Train Loss: 0.2510, Train Acc: 0.8889, Val Loss: 0.3880, Val Acc:
0.8500
Epoch 18: Train Loss: 0.2481, Train Acc: 0.8833, Val Loss: 0.3753, Val Acc:
0.8500
Epoch 19: Train Loss: 0.2420, Train Acc: 0.8944, Val Loss: 0.3640, Val Acc:
0.8500
Epoch 20: Train Loss: 0.2331, Train Acc: 0.8944, Val Loss: 0.3560, Val Acc:
0.8500
Epoch 21: Train Loss: 0.2285, Train Acc: 0.8944, Val Loss: 0.3449, Val Acc:
0.8500
Epoch 22: Train Loss: 0.2246, Train Acc: 0.8944, Val Loss: 0.3369, Val Acc:
0.8500
Epoch 23: Train Loss: 0.2188, Train Acc: 0.8944, Val Loss: 0.3271, Val Acc:
0.8500
Epoch 24: Train Loss: 0.2116, Train Acc: 0.9000, Val Loss: 0.3139, Val Acc:
0.8500
Epoch 25: Train Loss: 0.2049, Train Acc: 0.9111, Val Loss: 0.3062, Val Acc:
0.8500
Epoch 26: Train Loss: 0.1993, Train Acc: 0.9222, Val Loss: 0.3013, Val Acc:
0.8500
Epoch 27: Train Loss: 0.1986, Train Acc: 0.9167, Val Loss: 0.2841, Val Acc:
0.8500
Epoch 28: Train Loss: 0.1859, Train Acc: 0.9278, Val Loss: 0.2878, Val Acc:
0.8500
Epoch 29: Train Loss: 0.1836, Train Acc: 0.9333, Val Loss: 0.2623, Val Acc:
0.8500
Epoch 30: Train Loss: 0.1763, Train Acc: 0.9444, Val Loss: 0.2615, Val Acc:
0.8500
Epoch 31: Train Loss: 0.1794, Train Acc: 0.9333, Val Loss: 0.2476, Val Acc:
0.8500
Epoch 32: Train Loss: 0.1616, Train Acc: 0.9444, Val Loss: 0.2437, Val Acc:
0.8500
Epoch 33: Train Loss: 0.1550, Train Acc: 0.9389, Val Loss: 0.2418, Val Acc:
```

```
0.8500
Epoch 34: Train Loss: 0.1527, Train Acc: 0.9444, Val Loss: 0.2272, Val Acc:
0.8500
Epoch 35: Train Loss: 0.1490, Train Acc: 0.9444, Val Loss: 0.2275, Val Acc:
0.8500
Epoch 36: Train Loss: 0.1496, Train Acc: 0.9556, Val Loss: 0.1927, Val Acc:
0.8500
Epoch 37: Train Loss: 0.1474, Train Acc: 0.9500, Val Loss: 0.2211, Val Acc:
0.8500
Epoch 38: Train Loss: 0.1420, Train Acc: 0.9500, Val Loss: 0.1940, Val Acc:
0.8500
Epoch 39: Train Loss: 0.1324, Train Acc: 0.9500, Val Loss: 0.2188, Val Acc:
0.8500
Epoch 40: Train Loss: 0.1281, Train Acc: 0.9611, Val Loss: 0.1913, Val Acc:
0.8500
Epoch 41: Train Loss: 0.1177, Train Acc: 0.9500, Val Loss: 0.1919, Val Acc:
0.8500
Epoch 42: Train Loss: 0.1170, Train Acc: 0.9611, Val Loss: 0.1786, Val Acc:
0.8500
Epoch 43: Train Loss: 0.1148, Train Acc: 0.9667, Val Loss: 0.1924, Val Acc:
0.8500
Epoch 44: Train Loss: 0.1164, Train Acc: 0.9500, Val Loss: 0.1698, Val Acc:
0.9000
Epoch 45: Train Loss: 0.1132, Train Acc: 0.9667, Val Loss: 0.1823, Val Acc:
0.8500
Epoch 46: Train Loss: 0.1082, Train Acc: 0.9611, Val Loss: 0.1645, Val Acc:
0.9000
Epoch 47: Train Loss: 0.1027, Train Acc: 0.9667, Val Loss: 0.1727, Val Acc:
0.9000
Epoch 48: Train Loss: 0.0980, Train Acc: 0.9611, Val Loss: 0.1554, Val Acc:
0.9000
Epoch 49: Train Loss: 0.0921, Train Acc: 0.9667, Val Loss: 0.1589, Val Acc:
0.9000
Epoch 50: Train Loss: 0.0866, Train Acc: 0.9611, Val Loss: 0.1451, Val Acc:
0.9000
Epoch 51: Train Loss: 0.0814, Train Acc: 0.9667, Val Loss: 0.1501, Val Acc:
0.9500
Epoch 52: Train Loss: 0.0811, Train Acc: 0.9667, Val Loss: 0.1364, Val Acc:
0.9000
Epoch 53: Train Loss: 0.0782, Train Acc: 0.9667, Val Loss: 0.1457, Val Acc:
0.9500
Epoch 54: Train Loss: 0.0778, Train Acc: 0.9667, Val Loss: 0.1277, Val Acc:
0.9500
Epoch 55: Train Loss: 0.0746, Train Acc: 0.9667, Val Loss: 0.1297, Val Acc:
0.9500
Epoch 56: Train Loss: 0.0708, Train Acc: 0.9667, Val Loss: 0.1173, Val Acc:
1.0000
Epoch 57: Train Loss: 0.0657, Train Acc: 0.9833, Val Loss: 0.1244, Val Acc:
```

```
0.9500
Epoch 58: Train Loss: 0.0636, Train Acc: 0.9833, Val Loss: 0.1091, Val Acc:
1.0000
Epoch 59: Train Loss: 0.0621, Train Acc: 0.9833, Val Loss: 0.1256, Val Acc:
0.9500
Epoch 60: Train Loss: 0.0604, Train Acc: 0.9833, Val Loss: 0.1043, Val Acc:
1.0000
Epoch 61: Train Loss: 0.0551, Train Acc: 0.9833, Val Loss: 0.1151, Val Acc:
0.9500
Epoch 62: Train Loss: 0.0557, Train Acc: 0.9889, Val Loss: 0.1010, Val Acc:
1.0000
Epoch 63: Train Loss: 0.0528, Train Acc: 0.9889, Val Loss: 0.1089, Val Acc:
0.9500
Epoch 64: Train Loss: 0.0501, Train Acc: 0.9889, Val Loss: 0.0907, Val Acc:
1.0000
Epoch 65: Train Loss: 0.0499, Train Acc: 1.0000, Val Loss: 0.1084, Val Acc:
0.9500
Epoch 66: Train Loss: 0.0511, Train Acc: 0.9889, Val Loss: 0.0923, Val Acc:
1.0000
Epoch 67: Train Loss: 0.0465, Train Acc: 0.9944, Val Loss: 0.1013, Val Acc:
0.9500
Epoch 68: Train Loss: 0.0442, Train Acc: 0.9889, Val Loss: 0.0850, Val Acc:
1.0000
Epoch 69: Train Loss: 0.0414, Train Acc: 0.9889, Val Loss: 0.0860, Val Acc:
1.0000
Epoch 70: Train Loss: 0.0409, Train Acc: 1.0000, Val Loss: 0.0963, Val Acc:
0.9500
Epoch 71: Train Loss: 0.0389, Train Acc: 0.9944, Val Loss: 0.0756, Val Acc:
1.0000
Epoch 72: Train Loss: 0.0365, Train Acc: 1.0000, Val Loss: 0.0870, Val Acc:
0.9500
Epoch 73: Train Loss: 0.0363, Train Acc: 1.0000, Val Loss: 0.0738, Val Acc:
1.0000
Epoch 74: Train Loss: 0.0337, Train Acc: 1.0000, Val Loss: 0.0815, Val Acc:
1.0000
Epoch 75: Train Loss: 0.0322, Train Acc: 1.0000, Val Loss: 0.0675, Val Acc:
1.0000
Epoch 76: Train Loss: 0.0313, Train Acc: 1.0000, Val Loss: 0.0747, Val Acc:
1.0000
Epoch 77: Train Loss: 0.0317, Train Acc: 1.0000, Val Loss: 0.0657, Val Acc:
1.0000
Epoch 78: Train Loss: 0.0298, Train Acc: 1.0000, Val Loss: 0.0775, Val Acc:
1.0000
Epoch 79: Train Loss: 0.0287, Train Acc: 1.0000, Val Loss: 0.0607, Val Acc:
1.0000
Epoch 80: Train Loss: 0.0276, Train Acc: 1.0000, Val Loss: 0.0711, Val Acc:
1.0000
Epoch 81: Train Loss: 0.0276, Train Acc: 1.0000, Val Loss: 0.0619, Val Acc:
```

```
1.0000
Epoch 82: Train Loss: 0.0258, Train Acc: 1.0000, Val Loss: 0.0654, Val Acc:
1.0000
Epoch 83: Train Loss: 0.0248, Train Acc: 1.0000, Val Loss: 0.0615, Val Acc:
1.0000
Epoch 84: Train Loss: 0.0235, Train Acc: 1.0000, Val Loss: 0.0553, Val Acc:
1.0000
Epoch 85: Train Loss: 0.0243, Train Acc: 1.0000, Val Loss: 0.0607, Val Acc:
1.0000
Epoch 86: Train Loss: 0.0225, Train Acc: 1.0000, Val Loss: 0.0557, Val Acc:
1.0000
Epoch 87: Train Loss: 0.0217, Train Acc: 1.0000, Val Loss: 0.0572, Val Acc:
1.0000
Epoch 88: Train Loss: 0.0209, Train Acc: 1.0000, Val Loss: 0.0522, Val Acc:
1.0000
Epoch 89: Train Loss: 0.0214, Train Acc: 1.0000, Val Loss: 0.0527, Val Acc:
1.0000
Epoch 90: Train Loss: 0.0199, Train Acc: 1.0000, Val Loss: 0.0553, Val Acc:
1.0000
Epoch 91: Train Loss: 0.0193, Train Acc: 1.0000, Val Loss: 0.0505, Val Acc:
1.0000
Epoch 92: Train Loss: 0.0187, Train Acc: 1.0000, Val Loss: 0.0506, Val Acc:
1.0000
Epoch 93: Train Loss: 0.0191, Train Acc: 1.0000, Val Loss: 0.0491, Val Acc:
1.0000
Epoch 94: Train Loss: 0.0179, Train Acc: 1.0000, Val Loss: 0.0501, Val Acc:
1.0000
Epoch 95: Train Loss: 0.0171, Train Acc: 1.0000, Val Loss: 0.0483, Val Acc:
1.0000
Epoch 96: Train Loss: 0.0166, Train Acc: 1.0000, Val Loss: 0.0457, Val Acc:
1.0000
Epoch 97: Train Loss: 0.0171, Train Acc: 1.0000, Val Loss: 0.0461, Val Acc:
1.0000
Epoch 98: Train Loss: 0.0163, Train Acc: 1.0000, Val Loss: 0.0472, Val Acc:
1.0000
Epoch 99: Train Loss: 0.0155, Train Acc: 1.0000, Val Loss: 0.0448, Val Acc:
1.0000
Epoch 100: Train Loss: 0.0153, Train Acc: 1.0000, Val Loss: 0.0431, Val Acc:
1.0000
```

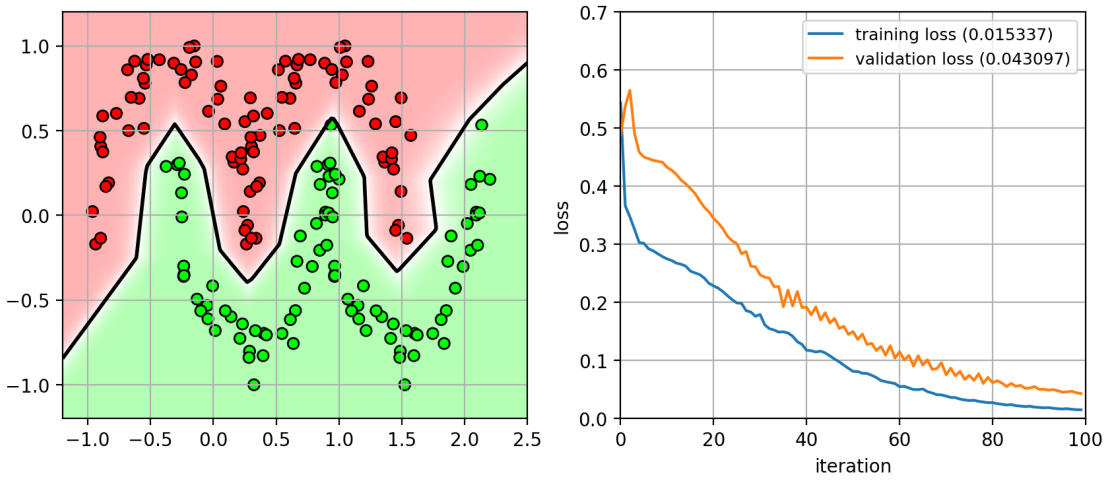
[24]:
```
summary_nn(model)
```

```
MyModel(
  (network): Sequential(
    (0): Linear(in_features=2, out_features=40, bias=True)
    (1): ReLU()
    (2): Linear(in_features=40, out_features=2, bias=True)
  )
```

```
)
Param: network.0.weight Shape: [40, 2]  Numel: 80
Param: network.0.bias   Shape: [40]     Numel: 40
Param: network.2.weight Shape: [2, 40]  Numel: 80
Param: network.2.bias   Shape: [2]      Numel: 2
Total Numel = 202
```

[25]:
```
nnfig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plot_nn(model, axbox2, trainX2, trainY2)
plt.subplot(1,2,2)
plot_history(history)
plt.close()

nnfig
```

[25]:



- More hidden layers:
  - input (2D) -> 16 nodes -> 16 nodes -> output (2D)

[26]:
```
model = MyModel(input_size=2, output_size=2, hidden_sizes=[16, 16])  # MLP model
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True) ␣
 ↪# optimizer
train_loader = get_dataloader(trainX2, trainY2)  # data loaders
val_loader = get_dataloader(valX2, valY2)
# train the model
history = train_model(model, criterion, train_loader, val_loader,␣
 ↪total_epoch=100, patience=50)
```

```
Epoch 1: Train Loss: 0.6153, Train Acc: 0.7389, Val Loss: 0.5550, Val Acc:
0.7000
Epoch 2: Train Loss: 0.3982, Train Acc: 0.8667, Val Loss: 0.4936, Val Acc:
```

```
0.8000
Epoch 3: Train Loss: 0.3542, Train Acc: 0.8500, Val Loss: 0.5445, Val Acc:
0.8000
Epoch 4: Train Loss: 0.3557, Train Acc: 0.8611, Val Loss: 0.5140, Val Acc:
0.8500
Epoch 5: Train Loss: 0.3206, Train Acc: 0.8333, Val Loss: 0.4709, Val Acc:
0.8000
Epoch 6: Train Loss: 0.3078, Train Acc: 0.8444, Val Loss: 0.4589, Val Acc:
0.8000
Epoch 7: Train Loss: 0.3065, Train Acc: 0.8611, Val Loss: 0.4617, Val Acc:
0.8000
Epoch 8: Train Loss: 0.3043, Train Acc: 0.8500, Val Loss: 0.4640, Val Acc:
0.8000
Epoch 9: Train Loss: 0.3011, Train Acc: 0.8444, Val Loss: 0.4645, Val Acc:
0.8000
Epoch 10: Train Loss: 0.2975, Train Acc: 0.8389, Val Loss: 0.4613, Val Acc:
0.8500
Epoch 11: Train Loss: 0.2940, Train Acc: 0.8500, Val Loss: 0.4555, Val Acc:
0.8500
Epoch 12: Train Loss: 0.2913, Train Acc: 0.8500, Val Loss: 0.4516, Val Acc:
0.8500
Epoch 13: Train Loss: 0.2892, Train Acc: 0.8500, Val Loss: 0.4490, Val Acc:
0.8500
Epoch 14: Train Loss: 0.2861, Train Acc: 0.8500, Val Loss: 0.4480, Val Acc:
0.8500
Epoch 15: Train Loss: 0.2833, Train Acc: 0.8667, Val Loss: 0.4436, Val Acc:
0.8500
Epoch 16: Train Loss: 0.2803, Train Acc: 0.8722, Val Loss: 0.4382, Val Acc:
0.8500
Epoch 17: Train Loss: 0.2780, Train Acc: 0.8667, Val Loss: 0.4303, Val Acc:
0.8500
Epoch 18: Train Loss: 0.2740, Train Acc: 0.8667, Val Loss: 0.4248, Val Acc:
0.8500
Epoch 19: Train Loss: 0.2703, Train Acc: 0.8667, Val Loss: 0.4177, Val Acc:
0.8500
Epoch 20: Train Loss: 0.2658, Train Acc: 0.8722, Val Loss: 0.4114, Val Acc:
0.8500
Epoch 21: Train Loss: 0.2616, Train Acc: 0.8778, Val Loss: 0.4027, Val Acc:
0.8500
Epoch 22: Train Loss: 0.2565, Train Acc: 0.8778, Val Loss: 0.3955, Val Acc:
0.8500
Epoch 23: Train Loss: 0.2524, Train Acc: 0.8722, Val Loss: 0.3828, Val Acc:
0.8500
Epoch 24: Train Loss: 0.2458, Train Acc: 0.8778, Val Loss: 0.3687, Val Acc:
0.8500
Epoch 25: Train Loss: 0.2397, Train Acc: 0.8778, Val Loss: 0.3604, Val Acc:
0.8500
Epoch 26: Train Loss: 0.2357, Train Acc: 0.8833, Val Loss: 0.3474, Val Acc:
```

```
0.8500
Epoch 27: Train Loss: 0.2303, Train Acc: 0.8778, Val Loss: 0.3238, Val Acc:
0.8500
Epoch 28: Train Loss: 0.2191, Train Acc: 0.8833, Val Loss: 0.3161, Val Acc:
0.8500
Epoch 29: Train Loss: 0.2176, Train Acc: 0.8944, Val Loss: 0.3041, Val Acc:
0.8500
Epoch 30: Train Loss: 0.2080, Train Acc: 0.9056, Val Loss: 0.3067, Val Acc:
0.8500
Epoch 31: Train Loss: 0.1975, Train Acc: 0.9056, Val Loss: 0.2725, Val Acc:
0.8500
Epoch 32: Train Loss: 0.1815, Train Acc: 0.9278, Val Loss: 0.2712, Val Acc:
0.8500
Epoch 33: Train Loss: 0.1762, Train Acc: 0.9278, Val Loss: 0.2531, Val Acc:
0.8500
Epoch 34: Train Loss: 0.1561, Train Acc: 0.9389, Val Loss: 0.2205, Val Acc:
0.8500
Epoch 35: Train Loss: 0.1444, Train Acc: 0.9444, Val Loss: 0.2262, Val Acc:
0.9000
Epoch 36: Train Loss: 0.1499, Train Acc: 0.9500, Val Loss: 0.2010, Val Acc:
0.9000
Epoch 37: Train Loss: 0.1176, Train Acc: 0.9556, Val Loss: 0.2529, Val Acc:
0.8500
Epoch 38: Train Loss: 0.1538, Train Acc: 0.9389, Val Loss: 0.1754, Val Acc:
0.9000
Epoch 39: Train Loss: 0.1062, Train Acc: 0.9611, Val Loss: 0.1505, Val Acc:
0.9500
Epoch 40: Train Loss: 0.0771, Train Acc: 0.9889, Val Loss: 0.1770, Val Acc:
0.9500
Epoch 41: Train Loss: 0.0733, Train Acc: 0.9889, Val Loss: 0.0944, Val Acc:
1.0000
Epoch 42: Train Loss: 0.0519, Train Acc: 0.9889, Val Loss: 0.1233, Val Acc:
0.9500
Epoch 43: Train Loss: 0.0492, Train Acc: 0.9889, Val Loss: 0.0872, Val Acc:
1.0000
Epoch 44: Train Loss: 0.0387, Train Acc: 0.9944, Val Loss: 0.1281, Val Acc:
0.9500
Epoch 45: Train Loss: 0.0504, Train Acc: 0.9889, Val Loss: 0.0637, Val Acc:
1.0000
Epoch 46: Train Loss: 0.0286, Train Acc: 1.0000, Val Loss: 0.1044, Val Acc:
0.9500
Epoch 47: Train Loss: 0.0341, Train Acc: 0.9889, Val Loss: 0.0607, Val Acc:
1.0000
Epoch 48: Train Loss: 0.0287, Train Acc: 1.0000, Val Loss: 0.1044, Val Acc:
0.9500
Epoch 49: Train Loss: 0.0326, Train Acc: 0.9889, Val Loss: 0.0582, Val Acc:
1.0000
Epoch 50: Train Loss: 0.0333, Train Acc: 0.9944, Val Loss: 0.0962, Val Acc:
```

```
0.9500
Epoch 51: Train Loss: 0.0288, Train Acc: 0.9889, Val Loss: 0.0543, Val Acc:
1.0000
Epoch 52: Train Loss: 0.0170, Train Acc: 1.0000, Val Loss: 0.0560, Val Acc:
1.0000
Epoch 53: Train Loss: 0.0169, Train Acc: 0.9889, Val Loss: 0.0407, Val Acc:
1.0000
Epoch 54: Train Loss: 0.0128, Train Acc: 1.0000, Val Loss: 0.0401, Val Acc:
1.0000
Epoch 55: Train Loss: 0.0107, Train Acc: 1.0000, Val Loss: 0.0371, Val Acc:
1.0000
Epoch 56: Train Loss: 0.0098, Train Acc: 1.0000, Val Loss: 0.0374, Val Acc:
1.0000
Epoch 57: Train Loss: 0.0096, Train Acc: 1.0000, Val Loss: 0.0357, Val Acc:
1.0000
Epoch 58: Train Loss: 0.0092, Train Acc: 1.0000, Val Loss: 0.0313, Val Acc:
1.0000
Epoch 59: Train Loss: 0.0075, Train Acc: 1.0000, Val Loss: 0.0304, Val Acc:
1.0000
Epoch 60: Train Loss: 0.0065, Train Acc: 1.0000, Val Loss: 0.0253, Val Acc:
1.0000
Epoch 61: Train Loss: 0.0073, Train Acc: 1.0000, Val Loss: 0.0265, Val Acc:
1.0000
Epoch 62: Train Loss: 0.0070, Train Acc: 1.0000, Val Loss: 0.0242, Val Acc:
1.0000
Epoch 63: Train Loss: 0.0063, Train Acc: 1.0000, Val Loss: 0.0297, Val Acc:
1.0000
Epoch 64: Train Loss: 0.0057, Train Acc: 1.0000, Val Loss: 0.0236, Val Acc:
1.0000
Epoch 65: Train Loss: 0.0059, Train Acc: 1.0000, Val Loss: 0.0219, Val Acc:
1.0000
Epoch 66: Train Loss: 0.0056, Train Acc: 1.0000, Val Loss: 0.0218, Val Acc:
1.0000
Epoch 67: Train Loss: 0.0050, Train Acc: 1.0000, Val Loss: 0.0232, Val Acc:
1.0000
Epoch 68: Train Loss: 0.0049, Train Acc: 1.0000, Val Loss: 0.0220, Val Acc:
1.0000
Epoch 69: Train Loss: 0.0048, Train Acc: 1.0000, Val Loss: 0.0186, Val Acc:
1.0000
Epoch 70: Train Loss: 0.0043, Train Acc: 1.0000, Val Loss: 0.0198, Val Acc:
1.0000
Epoch 71: Train Loss: 0.0038, Train Acc: 1.0000, Val Loss: 0.0196, Val Acc:
1.0000
Epoch 72: Train Loss: 0.0045, Train Acc: 1.0000, Val Loss: 0.0186, Val Acc:
1.0000
Epoch 73: Train Loss: 0.0040, Train Acc: 1.0000, Val Loss: 0.0187, Val Acc:
1.0000
Epoch 74: Train Loss: 0.0035, Train Acc: 1.0000, Val Loss: 0.0187, Val Acc:
```

```
1.0000
Epoch 75: Train Loss: 0.0041, Train Acc: 1.0000, Val Loss: 0.0185, Val Acc:
1.0000
Epoch 76: Train Loss: 0.0035, Train Acc: 1.0000, Val Loss: 0.0179, Val Acc:
1.0000
Epoch 77: Train Loss: 0.0032, Train Acc: 1.0000, Val Loss: 0.0172, Val Acc:
1.0000
Epoch 78: Train Loss: 0.0029, Train Acc: 1.0000, Val Loss: 0.0174, Val Acc:
1.0000
Epoch 79: Train Loss: 0.0035, Train Acc: 1.0000, Val Loss: 0.0165, Val Acc:
1.0000
Epoch 80: Train Loss: 0.0029, Train Acc: 1.0000, Val Loss: 0.0168, Val Acc:
1.0000
Epoch 81: Train Loss: 0.0026, Train Acc: 1.0000, Val Loss: 0.0171, Val Acc:
1.0000
Epoch 82: Train Loss: 0.0032, Train Acc: 1.0000, Val Loss: 0.0165, Val Acc:
1.0000
Epoch 83: Train Loss: 0.0026, Train Acc: 1.0000, Val Loss: 0.0163, Val Acc:
1.0000
Epoch 84: Train Loss: 0.0027, Train Acc: 1.0000, Val Loss: 0.0155, Val Acc:
1.0000
Epoch 85: Train Loss: 0.0023, Train Acc: 1.0000, Val Loss: 0.0159, Val Acc:
1.0000
Epoch 86: Train Loss: 0.0029, Train Acc: 1.0000, Val Loss: 0.0153, Val Acc:
1.0000
Epoch 87: Train Loss: 0.0023, Train Acc: 1.0000, Val Loss: 0.0154, Val Acc:
1.0000
Epoch 88: Train Loss: 0.0024, Train Acc: 1.0000, Val Loss: 0.0145, Val Acc:
1.0000
Epoch 89: Train Loss: 0.0021, Train Acc: 1.0000, Val Loss: 0.0149, Val Acc:
1.0000
Epoch 90: Train Loss: 0.0025, Train Acc: 1.0000, Val Loss: 0.0144, Val Acc:
1.0000
Epoch 91: Train Loss: 0.0021, Train Acc: 1.0000, Val Loss: 0.0146, Val Acc:
1.0000
Early stopping triggered
```

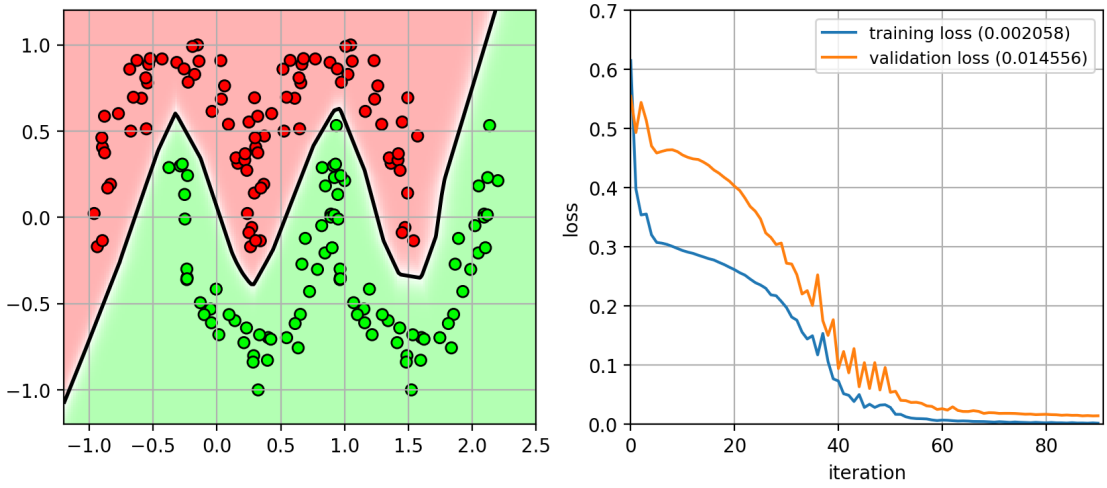
[27]:
```
summary_nn(model)
```

```
MyModel(
  (network): Sequential(
    (0): Linear(in_features=2, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=16, bias=True)
    (3): ReLU()
    (4): Linear(in_features=16, out_features=2, bias=True)
  )
)
```

```
Param: network.0.weight Shape: [16, 2]  Numel: 32
Param: network.0.bias   Shape: [16]     Numel: 16
Param: network.2.weight Shape: [16, 16] Numel: 256
Param: network.2.bias   Shape: [16]     Numel: 16
Param: network.4.weight Shape: [2, 16]  Numel: 32
Param: network.4.bias   Shape: [2]      Numel: 2
Total Numel = 354
```

[28]:
```
nnfig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plot_nn(model, axbox2, trainX2, trainY2)
plt.subplot(1,2,2)
plot_history(history)
plt.close()

nnfig
```

[28]:



18 We should use deeper networks...

- Less parameters than a 1 hidden-layer NN
  - but the number of parameters is still large
- Dataset is still too small.
- *Vanishing Gradient problem*
  - backprop recursively multiplies gradients
    - numerical values get smaller.
    - gradient signal is "washed" out the further back it travels.
- We'll see how to address these problems later.

19 Example on MNIST Dataset

- Images are 28x28, digits 0-9
  - 6,000 for training
  - 10,000 for testing

[29]:
```
def show_imgs(W_list, nc=10, highlight_green=None, highlight_red=None,␣
 ↪titles=None):
    nfilter = len(W_list)
    nr = (nfilter - 1) // nc + 1
    for i in range(nr):
        for j in range(nc):
            idx = i * nc + j
            if idx == nfilter:
                break
            plt.subplot(nr, nc, idx + 1)
            cur_W = W_list[idx]
            plt.imshow(cur_W,cmap='gray', interpolation='nearest')
            if titles is not None:
                plt.title(titles % idx)

            if ((highlight_green is not None) and highlight_green[idx]) or \
               ((highlight_red is not None) and highlight_red[idx]):
                ax = plt.gca()
                if highlight_green[idx]:
                    mycol = '#00FF00'
                else:
                    mycol = 'r'
                for S in ['bottom', 'top', 'right', 'left']:
                    ax.spines[S].set_color(mycol)
                    ax.spines[S].set_lw(2.0)
                ax.xaxis.set_ticks_position('none')
                ax.yaxis.set_ticks_position('none')
                ax.set_xticks([])
                ax.set_yticks([])
            else:
                plt.gca().set_axis_off()
```

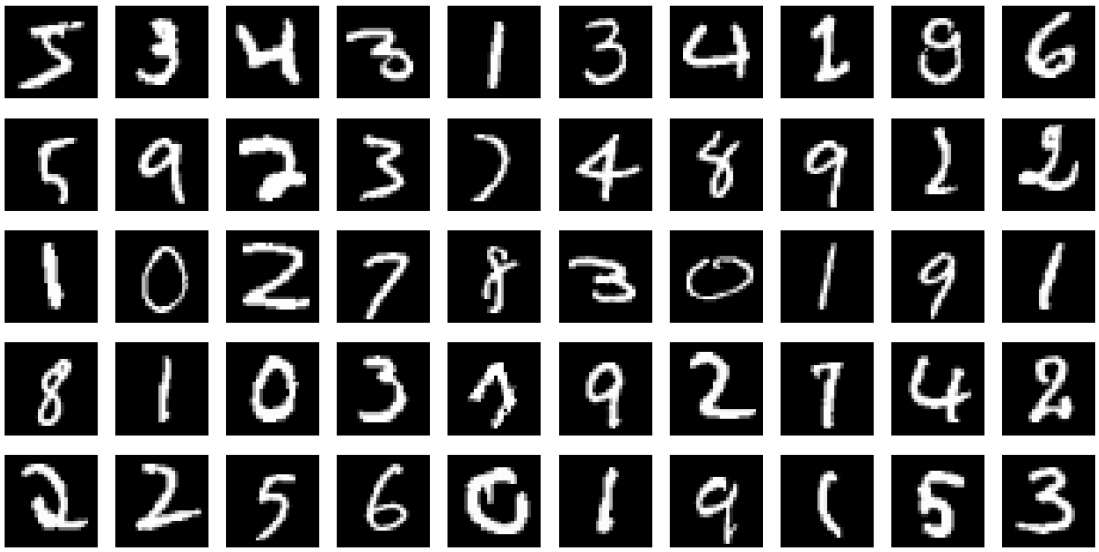
[30]:
```
import struct

def read_32int(f):
    return struct.unpack('>I', f.read(4))[0]
def read_img(img_path):
    with open(img_path, 'rb') as f:
        magic_num = read_32int(f)
        num_image = read_32int(f)
        n_row = read_32int(f)
        n_col = read_32int(f)
```

```
        #print 'num_image = {}; n_row = {}; n_col = {}'.format(num_image,␣
 ↪n_row, n_col)
        res = []
        npixel = n_row * n_col
        res_arr = fromfile(f, dtype='B')
        res_arr = res_arr.reshape((num_image, n_row, n_col), order='C')
        #print 'image data shape = {}'.format(res_arr.shape)
        return num_image, n_row, n_col, res_arr
def read_label(label_path):
    with open(label_path, 'rb') as f:
        magic_num = read_32int(f)
        num_label = read_32int(f)
        #print 'num_label = {}'.format(num_label)
        res_arr = fromfile(f, dtype='B')
        #print res_arr.shape
        #res_arr = res_arr.reshape((num_label, 1))
        res_arr = res_arr.ravel()
        #print 'label data shape = {}'.format(res_arr.shape)
        return num_label, res_arr
```

[31]:
```
n_train, nrow, ncol, trainimg = read_img('data/train-images.idx3-ubyte')
_, trainY = read_label('data/train-labels.idx1-ubyte')
n_test, _, _, testimg = read_img('data/t10k-images.idx3-ubyte')
_, testY = read_label('data/t10k-labels.idx1-ubyte')

# for demonstration we only use 10% of the training data
sample_index = range(0, trainimg.shape[0], 10)
trainimg  = trainimg[sample_index]
trainY    = trainY[sample_index]
print(trainimg.shape)
print(trainY.shape)
print(testimg.shape)
print(testY.shape)
```

```
(6000, 28, 28)
(6000,)
(10000, 28, 28)
(10000,)
```

[32]:
```
# Example images
plt.figure(figsize=(8,4))
show_imgs(trainimg[0:50])
```



20 Pre-processing

- Reshape images into vectors
- map to [0,1], then subtract the mean

[33]:
```
# Reshape the images to a vector
# and map the data to [0,1]
trainXraw = trainimg.reshape((len(trainimg), -1), order='C') / 255.0
testXraw = testimg.reshape((len(testimg), -1), order='C') / 255.0

# center the image data (but don't change variance)
scaler = preprocessing.StandardScaler(with_std=False)
trainX = scaler.fit_transform(trainXraw).astype(np.float32)
testX  = scaler.transform(testXraw).astype(np.float32)

print(trainX.shape)
print(trainY.shape)
```

```
(6000, 784)
(6000,)
```

- Generate a fixed validation set
  - use vtrainX for training and validX for validation

[34]:
```
# generate a fixed validation set using 10% of the training set
vtrainX, validX, vtrainY, validY = \
  model_selection.train_test_split(trainX, trainY,
  train_size=0.9, test_size=0.1, random_state=4487)
```

```
# validation data
validset  = (validX, validY)
```

21 MNIST - Logistic Regression (0-hidden layers)

- Training procedure
  - We specify the validation set so that it will be fixed when we change the `random_state` to randomly initialize the weights.
  - Train on the non-validation training data.
  - Use a larger batch size to speed up the algorithm

[35]:
```
def plot_history2(history):
    fig, ax1 = plt.subplots()
    ax1.plot(history['train_loss'], 'r', label="training loss ({:.6f})".
 ↪format(history['train_loss'][-1]))
    ax1.plot(history['val_loss'], 'r--', label="validation loss ({:.6f})".
 ↪format(history['val_loss'][-1]))
    ax1.grid(True)
    ax1.set_xlabel('iteration')
    ax1.legend(loc="best", fontsize=9)
    ax1.set_ylabel('loss', color='r')
    ax1.tick_params('y', colors='r')
    if 'train_acc' in history:
        ax2 = ax1.twinx()
        ax2.plot(history['train_acc'], 'b', label="training acc ({:.4f})".
 ↪format(history['train_acc'][-1]))
        ax2.plot(history['val_acc'], 'b--', label="validation acc ({:.4f})".
 ↪format(history['val_acc'][-1]))
        ax2.legend(loc="best", fontsize=9)
        ax2.set_ylabel('acc', color='b')
        ax2.tick_params('y', colors='b')
```

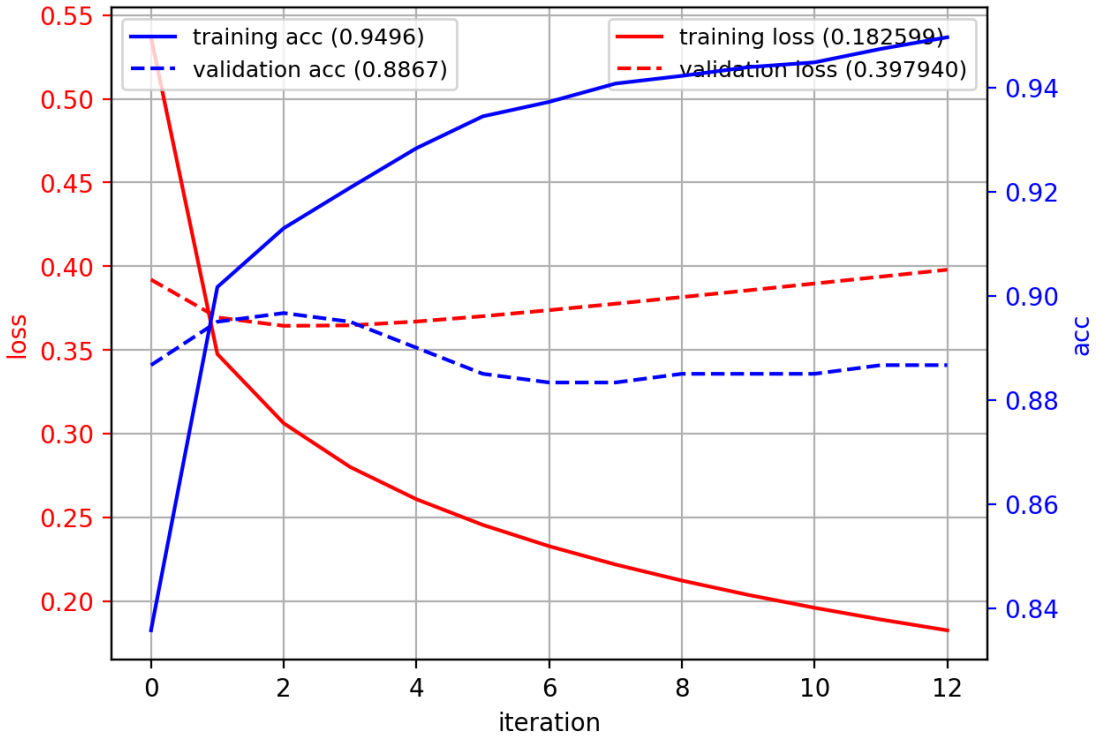
[36]:
```
model = MyModel(input_size=784, output_size=10, hidden_sizes=[])
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True) ␣
 ↪# optimizer
train_loader = get_dataloader(vtrainX, vtrainY)  # data loaders
val_loader = get_dataloader(validX, validY)
# train the model
history = train_model(model, criterion, train_loader, val_loader,␣
 ↪total_epoch=100, patience=10)
```

```
Epoch 1: Train Loss: 0.5367, Train Acc: 0.8357, Val Loss: 0.3920, Val Acc:
0.8867
Epoch 2: Train Loss: 0.3476, Train Acc: 0.9017, Val Loss: 0.3694, Val Acc:
0.8950
Epoch 3: Train Loss: 0.3062, Train Acc: 0.9130, Val Loss: 0.3644, Val Acc:
```

```
0.8967
Epoch 4: Train Loss: 0.2801, Train Acc: 0.9207, Val Loss: 0.3647, Val Acc:
0.8950
Epoch 5: Train Loss: 0.2609, Train Acc: 0.9283, Val Loss: 0.3670, Val Acc:
0.8900
Epoch 6: Train Loss: 0.2456, Train Acc: 0.9344, Val Loss: 0.3702, Val Acc:
0.8850
Epoch 7: Train Loss: 0.2328, Train Acc: 0.9372, Val Loss: 0.3738, Val Acc:
0.8833
Epoch 8: Train Loss: 0.2219, Train Acc: 0.9407, Val Loss: 0.3776, Val Acc:
0.8833
Epoch 9: Train Loss: 0.2123, Train Acc: 0.9422, Val Loss: 0.3816, Val Acc:
0.8850
Epoch 10: Train Loss: 0.2038, Train Acc: 0.9439, Val Loss: 0.3856, Val Acc:
0.8850
Epoch 11: Train Loss: 0.1960, Train Acc: 0.9448, Val Loss: 0.3897, Val Acc:
0.8850
Epoch 12: Train Loss: 0.1890, Train Acc: 0.9474, Val Loss: 0.3938, Val Acc:
0.8867
Epoch 13: Train Loss: 0.1826, Train Acc: 0.9496, Val Loss: 0.3979, Val Acc:
0.8867
Early stopping triggered
```

[37]:
```
summary_nn(model)
plot_history2(history)

with torch.no_grad():
    predY = model(torch.as_tensor(testX)).argmax(-1)
acc = metrics.accuracy_score(testY, predY)
print("test accuracy:", acc)
```

```
MyModel(
  (network): Sequential(
    (0): Linear(in_features=784, out_features=10, bias=True)
  )
)
Param: network.0.weight Shape: [10, 784]        Numel: 7840
Param: network.0.bias   Shape: [10]     Numel: 10
Total Numel = 7850
test accuracy: 0.8813
```

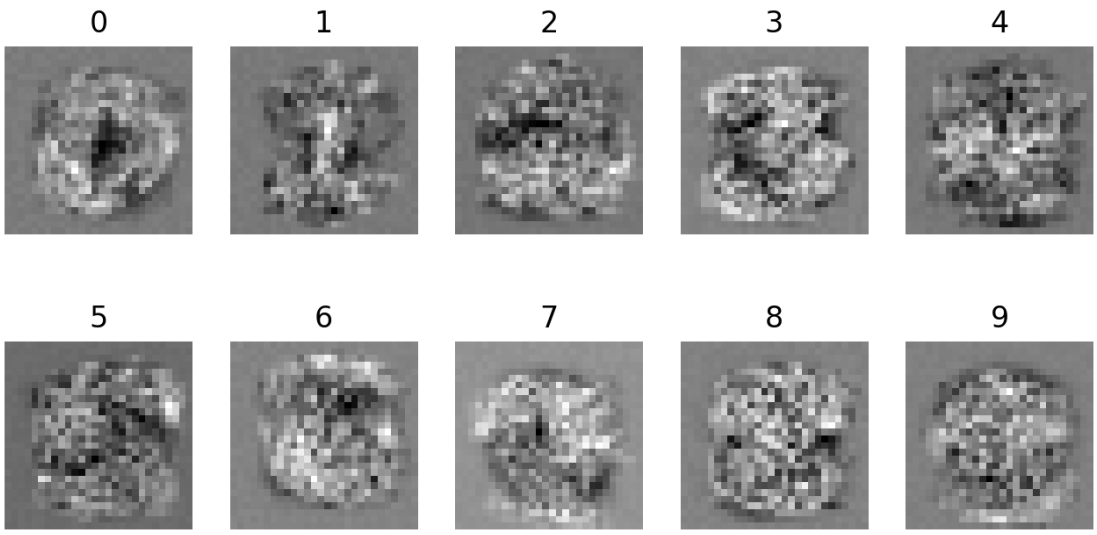


[38]:
```
params = model.network[0].weight.data
print(params)
```

```
tensor([[ 3.3546e-02, -3.4324e-02,  2.0939e-02,  …, -2.0905e-02,
          1.1012e-02, -3.1561e-02],
        [ 2.7129e-02,  3.1136e-03, -1.7196e-02,  …, -1.5344e-02,
         -2.0055e-02, -2.7852e-03],
        [ 2.9864e-02, -3.4973e-02, -1.7804e-02,  …,  1.1404e-02,
         -8.0777e-03, -5.1047e-05],
        …,
        [-1.3821e-02, -1.6467e-03, -2.6531e-03,  …,  2.2454e-02,
          1.5731e-02,  5.1474e-03],
        [-4.0478e-03,  2.0415e-02, -5.3243e-03,  …,  2.2182e-02,
         -2.7403e-02, -2.7426e-02],
        [-2.7465e-02, -8.7818e-03,  2.0570e-02,  …, -5.8698e-05,
         -1.5991e-02, -1.2174e-04]])
```

- Reshape the weights into an image
  - input images that match the weights will have high response for that class.

[39]:
```
filter_list = [w.view((28,28)) for w in params]
plt.figure(figsize=(8,4))
show_imgs(filter_list, nc=5, titles="%d")
```



22 MNIST - 1-hidden layer

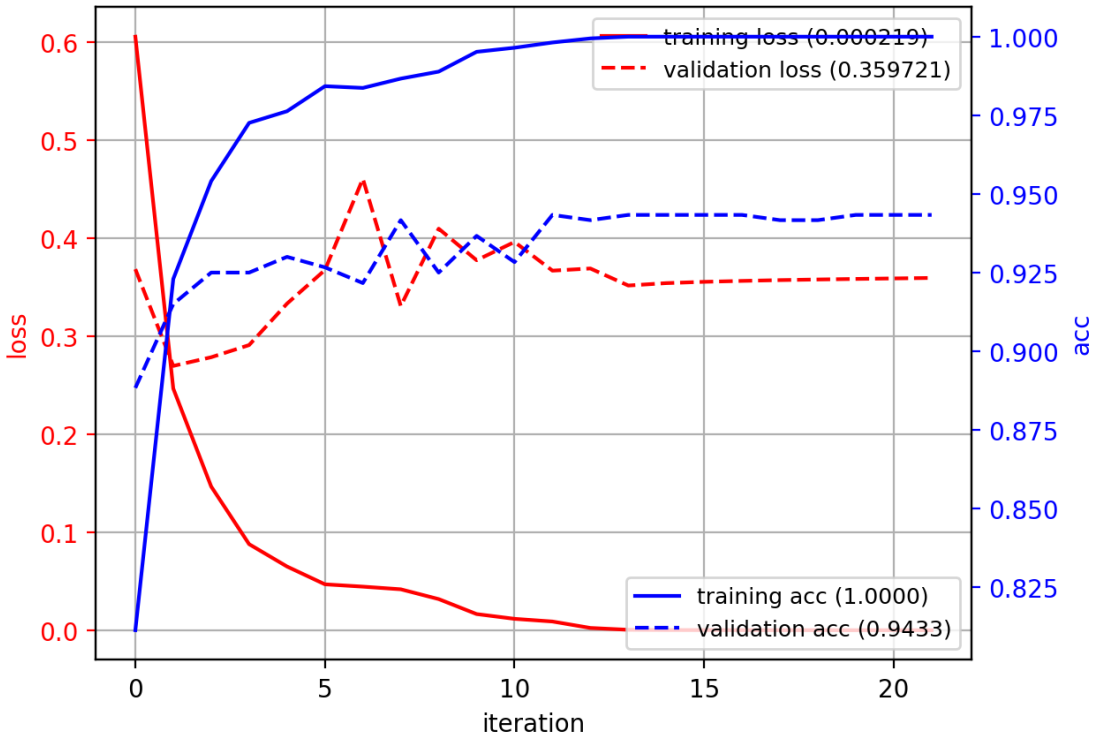
- Add 1 hidden layer with 50 ReLu nodes
  - each node is extracting a feature from the input image

[40]:
```
model = MyModel(input_size=784, output_size=10, hidden_sizes=[50])
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True) ␣
 ↪# optimizer
train_loader = get_dataloader(vtrainX, vtrainY)  # data loaders
val_loader = get_dataloader(validX, validY)
# train the model
history = train_model(model, criterion, train_loader, val_loader,␣
 ↪total_epoch=100, patience=10)
# summary and plot
summary_nn(model)
plot_history2(history)
# test
with torch.no_grad():
    predY = model(torch.as_tensor(testX)).argmax(-1)
acc = metrics.accuracy_score(testY, predY)
print("test accuracy:", acc)
```

```
Epoch 1: Train Loss: 0.6059, Train Acc: 0.8113, Val Loss: 0.3687, Val Acc:
0.8883
Epoch 2: Train Loss: 0.2468, Train Acc: 0.9230, Val Loss: 0.2699, Val Acc:
0.9150
Epoch 3: Train Loss: 0.1468, Train Acc: 0.9541, Val Loss: 0.2788, Val Acc:
0.9250
```

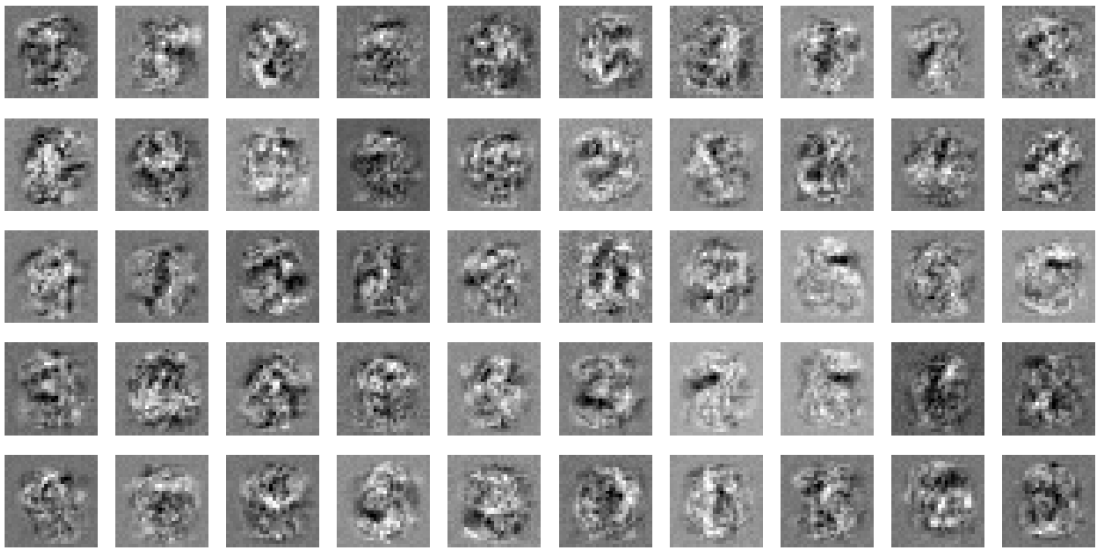
```
Epoch 4: Train Loss: 0.0880, Train Acc: 0.9726, Val Loss: 0.2912, Val Acc:
0.9250
Epoch 5: Train Loss: 0.0652, Train Acc: 0.9763, Val Loss: 0.3335, Val Acc:
0.9300
Epoch 6: Train Loss: 0.0471, Train Acc: 0.9843, Val Loss: 0.3678, Val Acc:
0.9267
Epoch 7: Train Loss: 0.0447, Train Acc: 0.9837, Val Loss: 0.4609, Val Acc:
0.9217
Epoch 8: Train Loss: 0.0420, Train Acc: 0.9867, Val Loss: 0.3310, Val Acc:
0.9417
Epoch 9: Train Loss: 0.0320, Train Acc: 0.9889, Val Loss: 0.4100, Val Acc:
0.9250
Epoch 10: Train Loss: 0.0167, Train Acc: 0.9952, Val Loss: 0.3778, Val Acc:
0.9367
Epoch 11: Train Loss: 0.0118, Train Acc: 0.9965, Val Loss: 0.3964, Val Acc:
0.9283
Epoch 12: Train Loss: 0.0091, Train Acc: 0.9981, Val Loss: 0.3672, Val Acc:
0.9433
Epoch 13: Train Loss: 0.0025, Train Acc: 0.9994, Val Loss: 0.3694, Val Acc:
0.9417
Epoch 14: Train Loss: 0.0007, Train Acc: 1.0000, Val Loss: 0.3520, Val Acc:
0.9433
Epoch 15: Train Loss: 0.0004, Train Acc: 1.0000, Val Loss: 0.3544, Val Acc:
0.9433
Epoch 16: Train Loss: 0.0004, Train Acc: 1.0000, Val Loss: 0.3558, Val Acc:
0.9433
Epoch 17: Train Loss: 0.0003, Train Acc: 1.0000, Val Loss: 0.3567, Val Acc:
0.9433
Epoch 18: Train Loss: 0.0003, Train Acc: 1.0000, Val Loss: 0.3575, Val Acc:
0.9417
Epoch 19: Train Loss: 0.0003, Train Acc: 1.0000, Val Loss: 0.3581, Val Acc:
0.9417
Epoch 20: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.3587, Val Acc:
0.9433
Epoch 21: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.3592, Val Acc:
0.9433
Epoch 22: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.3597, Val Acc:
0.9433
Early stopping triggered
MyModel(
  (network): Sequential(
    (0): Linear(in_features=784, out_features=50, bias=True)
    (1): ReLU()
    (2): Linear(in_features=50, out_features=10, bias=True)
  )
)
Param: network.0.weight Shape: [50, 784]        Numel: 39200
Param: network.0.bias   Shape: [50]     Numel: 50
```

```
Param: network.2.weight Shape: [10, 50] Numel: 500
Param: network.2.bias   Shape: [10]     Numel: 10
Total Numel = 39760
test accuracy: 0.9361
```



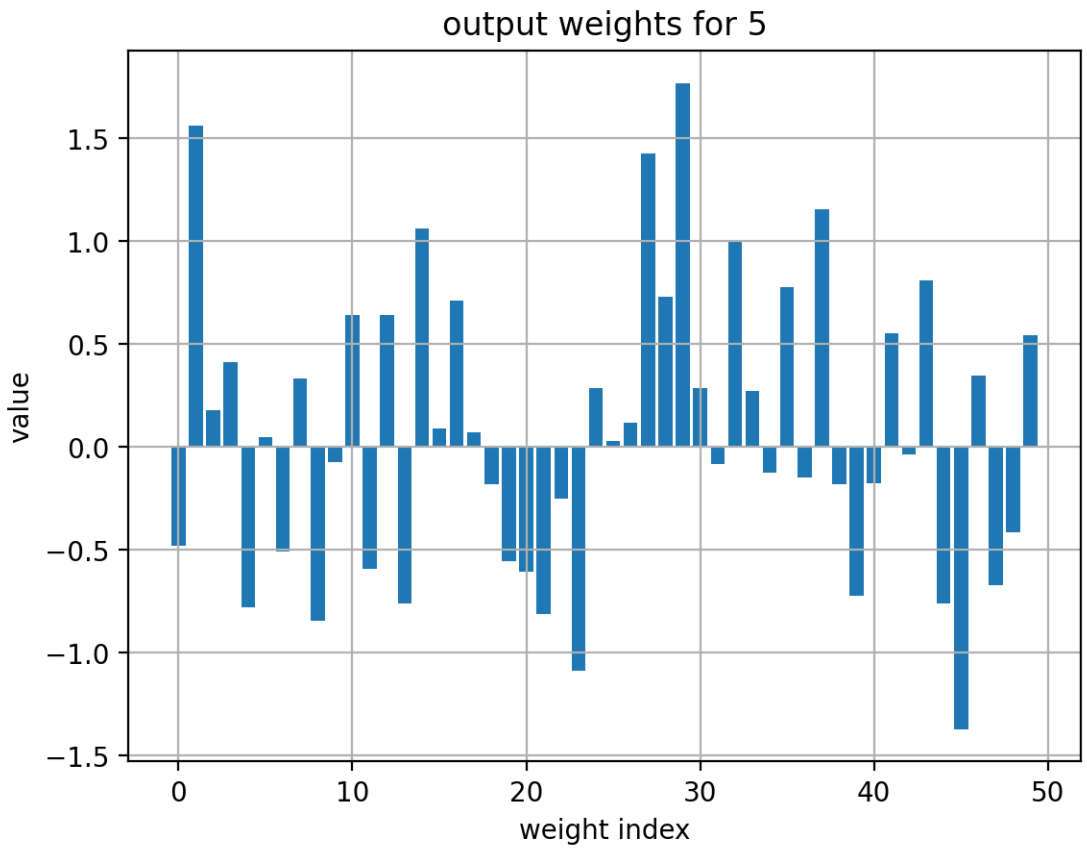
- Examine the weights of the hidden layer
  - $h_i = \sigma(\mathbf{w}_i^T \mathbf{x})$
  - each weight vector is a "pattern prototype" that the node will match
- The hidden nodes look for local structures:
  - oriented edges, curves, other local structures

[41]:
```
W = model.network[0].weight.data
filter_list = [w.view((28,28)) for w in W]
plt.figure(figsize=(8,4))
show_imgs(filter_list, nc=10)
```



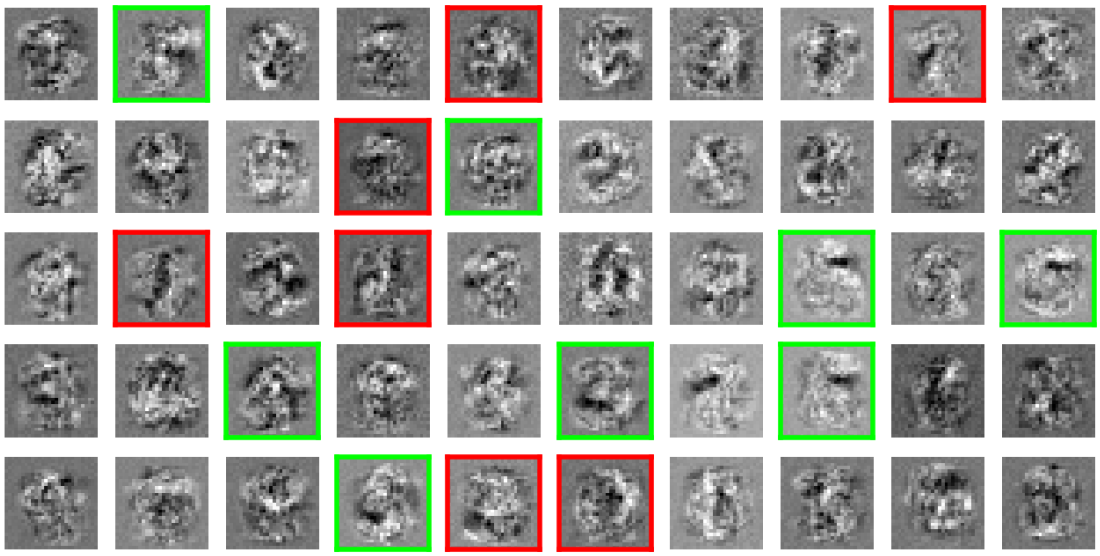
- Examine the weights of the 2nd layer (output)
  - $y_j = \sigma(\mathbf{w}_j^T \mathbf{h})$
  - recall the hidden-layer outputs h are always non-negative.
    - positive value in $\mathbf{w}_j \rightarrow$ class j should have j-th pattern
    - negative value in $\mathbf{w}_j \rightarrow$ class j shouldn't have j-th pattern

[42]:
```
W = model.network[2].weight.data.T  # model.network[1] is the relu
# print(W.shape)
d = 5
plt.bar(arange(0,W.shape[0]),W[:,d]); plt.grid(True);
plt.xlabel('weight index'); plt.ylabel('value')
plt.title('output weights for {}'.format(d));
```



- For "5", finds local image parts that correspond to 5
  - should have (green boxes):
    - horizontal line at top; semicircle on the bottom
  - shouldn't have (red boxes):
    - vertical line in top-right; verticle line in the middle

[43]:
```
plt.figure(figsize=(8,4))
show_imgs(filter_list, nc=10,
          highlight_green=(W[:,d]>0.75), # positive weights are green
          highlight_red=(W[:,d]<-0.75))  # negative weights are red
```



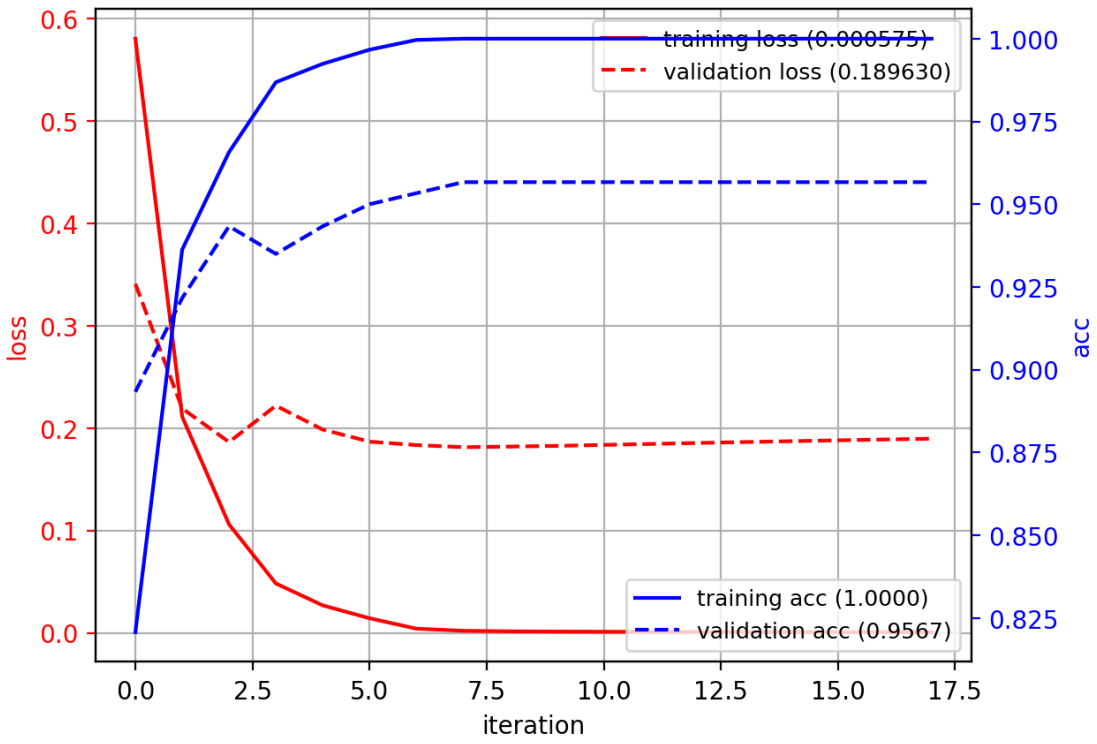
23 1 Hidden layer with more nodes

- hidden layer with 200 nodes

[44]:
```
model = MyModel(input_size=784, output_size=10, hidden_sizes=[200])
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True)
 ↪# optimizer
train_loader = get_dataloader(vtrainX, vtrainY)  # data loaders
val_loader = get_dataloader(validX, validY)
# train the model
history = train_model(model, criterion, train_loader, val_loader,
 ↪total_epoch=100, patience=10)
# summary and plot
summary_nn(model)
plot_history2(history)
# test
with torch.no_grad():
    predY = model(torch.as_tensor(testX)).argmax(-1)
acc = metrics.accuracy_score(testY, predY)
print("test accuracy:", acc)
```

```
Epoch 1: Train Loss: 0.5805, Train Acc: 0.8207, Val Loss: 0.3409, Val Acc:
0.8933
Epoch 2: Train Loss: 0.2112, Train Acc: 0.9363, Val Loss: 0.2192, Val Acc:
0.9217
Epoch 3: Train Loss: 0.1060, Train Acc: 0.9657, Val Loss: 0.1864, Val Acc:
0.9433
```

```
Epoch 4: Train Loss: 0.0482, Train Acc: 0.9869, Val Loss: 0.2219, Val Acc:
0.9350
Epoch 5: Train Loss: 0.0268, Train Acc: 0.9924, Val Loss: 0.1986, Val Acc:
0.9433
Epoch 6: Train Loss: 0.0142, Train Acc: 0.9967, Val Loss: 0.1868, Val Acc:
0.9500
Epoch 7: Train Loss: 0.0040, Train Acc: 0.9996, Val Loss: 0.1834, Val Acc:
0.9533
Epoch 8: Train Loss: 0.0020, Train Acc: 1.0000, Val Loss: 0.1814, Val Acc:
0.9567
Epoch 9: Train Loss: 0.0014, Train Acc: 1.0000, Val Loss: 0.1819, Val Acc:
0.9567
Epoch 10: Train Loss: 0.0012, Train Acc: 1.0000, Val Loss: 0.1827, Val Acc:
0.9567
Epoch 11: Train Loss: 0.0010, Train Acc: 1.0000, Val Loss: 0.1836, Val Acc:
0.9567
Epoch 12: Train Loss: 0.0009, Train Acc: 1.0000, Val Loss: 0.1845, Val Acc:
0.9567
Epoch 13: Train Loss: 0.0008, Train Acc: 1.0000, Val Loss: 0.1855, Val Acc:
0.9567
Epoch 14: Train Loss: 0.0008, Train Acc: 1.0000, Val Loss: 0.1864, Val Acc:
0.9567
Epoch 15: Train Loss: 0.0007, Train Acc: 1.0000, Val Loss: 0.1872, Val Acc:
0.9567
Epoch 16: Train Loss: 0.0007, Train Acc: 1.0000, Val Loss: 0.1881, Val Acc:
0.9567
Epoch 17: Train Loss: 0.0006, Train Acc: 1.0000, Val Loss: 0.1889, Val Acc:
0.9567
Epoch 18: Train Loss: 0.0006, Train Acc: 1.0000, Val Loss: 0.1896, Val Acc:
0.9567
Early stopping triggered
MyModel(
  (network): Sequential(
    (0): Linear(in_features=784, out_features=200, bias=True)
    (1): ReLU()
    (2): Linear(in_features=200, out_features=10, bias=True)
  )
)
Param: network.0.weight Shape: [200, 784]       Numel: 156800
Param: network.0.bias   Shape: [200]    Numel: 200
Param: network.2.weight Shape: [10, 200]        Numel: 2000
Param: network.2.bias   Shape: [10]     Numel: 10
Total Numel = 159010
test accuracy: 0.945
```

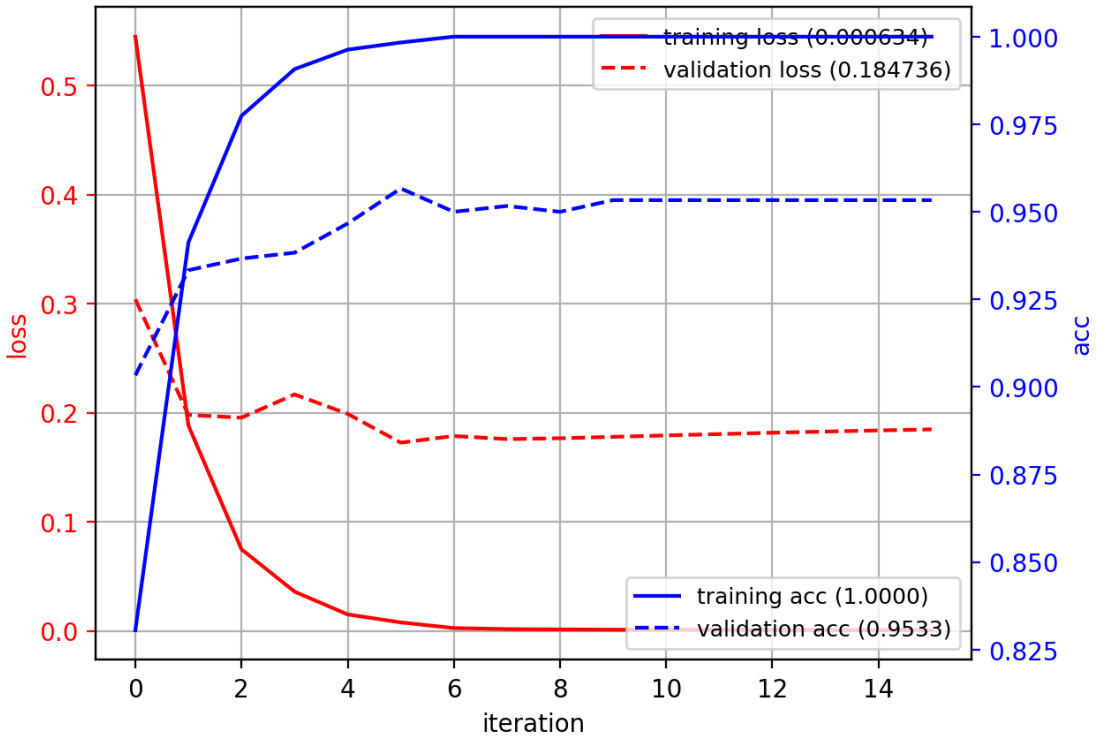


- hidden layer with 1000 nodes

[45]:
```
model = MyModel(input_size=784, output_size=10, hidden_sizes=[1000])
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True)
 ↪# optimizer
train_loader = get_dataloader(vtrainX, vtrainY)  # data loaders
val_loader = get_dataloader(validX, validY)
# train the model
history = train_model(model, criterion, train_loader, val_loader,
 ↪total_epoch=100, patience=10)
# summary and plot
summary_nn(model)
plot_history2(history)
# test
with torch.no_grad():
    predY = model(torch.as_tensor(testX)).argmax(-1)
acc = metrics.accuracy_score(testY, predY)
print("test accuracy:", acc)
```

```
Epoch 1: Train Loss: 0.5448, Train Acc: 0.8306, Val Loss: 0.3040, Val Acc:
0.9033
Epoch 2: Train Loss: 0.1883, Train Acc: 0.9413, Val Loss: 0.1980, Val Acc:
0.9333
```

```
Epoch 3: Train Loss: 0.0748, Train Acc: 0.9774, Val Loss: 0.1955, Val Acc:
0.9367
Epoch 4: Train Loss: 0.0360, Train Acc: 0.9907, Val Loss: 0.2168, Val Acc:
0.9383
Epoch 5: Train Loss: 0.0151, Train Acc: 0.9963, Val Loss: 0.1990, Val Acc:
0.9467
Epoch 6: Train Loss: 0.0078, Train Acc: 0.9983, Val Loss: 0.1726, Val Acc:
0.9567
Epoch 7: Train Loss: 0.0026, Train Acc: 1.0000, Val Loss: 0.1787, Val Acc:
0.9500
Epoch 8: Train Loss: 0.0016, Train Acc: 1.0000, Val Loss: 0.1758, Val Acc:
0.9517
Epoch 9: Train Loss: 0.0013, Train Acc: 1.0000, Val Loss: 0.1766, Val Acc:
0.9500
Epoch 10: Train Loss: 0.0011, Train Acc: 1.0000, Val Loss: 0.1779, Val Acc:
0.9533
Epoch 11: Train Loss: 0.0010, Train Acc: 1.0000, Val Loss: 0.1792, Val Acc:
0.9533
Epoch 12: Train Loss: 0.0009, Train Acc: 1.0000, Val Loss: 0.1804, Val Acc:
0.9533
Epoch 13: Train Loss: 0.0008, Train Acc: 1.0000, Val Loss: 0.1817, Val Acc:
0.9533
Epoch 14: Train Loss: 0.0007, Train Acc: 1.0000, Val Loss: 0.1828, Val Acc:
0.9533
Epoch 15: Train Loss: 0.0007, Train Acc: 1.0000, Val Loss: 0.1838, Val Acc:
0.9533
Epoch 16: Train Loss: 0.0006, Train Acc: 1.0000, Val Loss: 0.1847, Val Acc:
0.9533
Early stopping triggered
MyModel(
  (network): Sequential(
    (0): Linear(in_features=784, out_features=1000, bias=True)
    (1): ReLU()
    (2): Linear(in_features=1000, out_features=10, bias=True)
  )
)
Param: network.0.weight Shape: [1000, 784]      Numel: 784000
Param: network.0.bias   Shape: [1000]   Numel: 1000
Param: network.2.weight Shape: [10, 1000]       Numel: 10000
Param: network.2.bias   Shape: [10]     Numel: 10
Total Numel = 795010
test accuracy: 0.9487
```



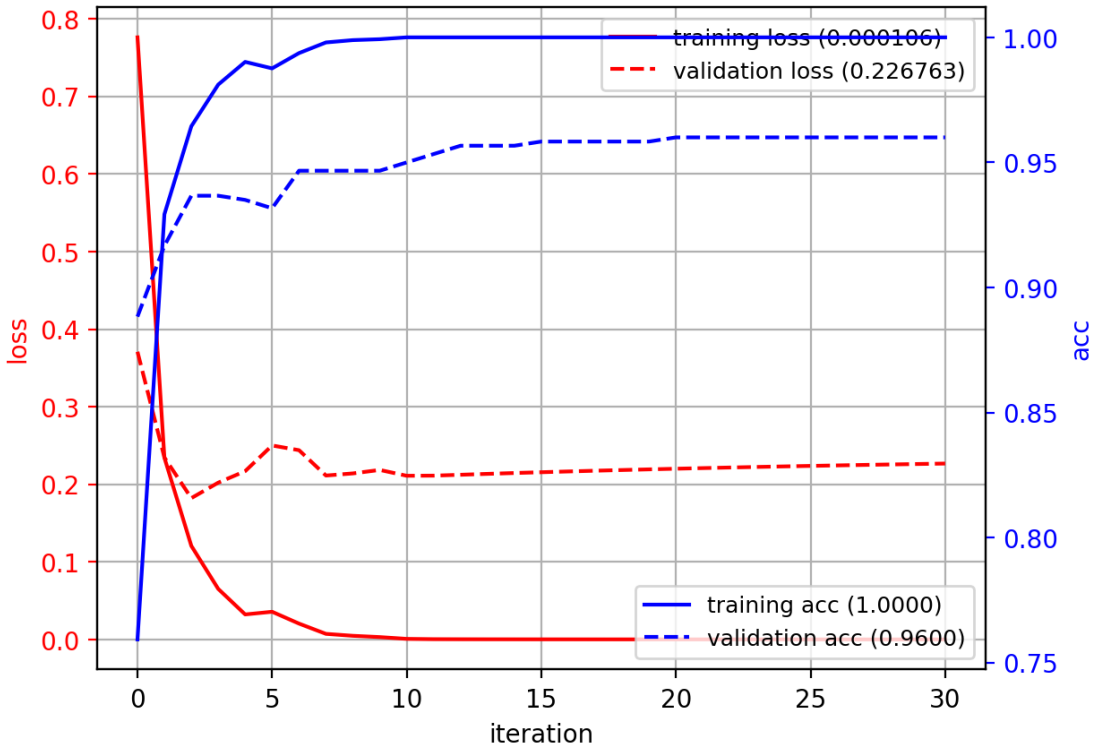
- 2 hidden layers
  - input (28x28) -> 500 nodes -> 500 nodes -> output
  - Slightly better

[46]:
```
model = MyModel(input_size=784, output_size=10, hidden_sizes=[500, 500])
criterion = nn.CrossEntropyLoss()  # loss function
optimizer = optim.SGD(model.parameters(), lr=0.05, momentum=0.9, nesterov=True)␣
 ↪ # note: need to tune the learning rate!
train_loader = get_dataloader(vtrainX, vtrainY)  # data loaders
val_loader = get_dataloader(validX, validY)
# train the model
history = train_model(model, criterion, train_loader, val_loader,␣
 ↪total_epoch=100, patience=10)
# summary and plot
summary_nn(model)
plot_history2(history)
# test
with torch.no_grad():
    predY = model(torch.as_tensor(testX)).argmax(-1)
acc = metrics.accuracy_score(testY, predY)
print("test accuracy:", acc)
```

```
Epoch 1: Train Loss: 0.7761, Train Acc: 0.7593, Val Loss: 0.3708, Val Acc:
0.8883
```

```
Epoch 2: Train Loss: 0.2343, Train Acc: 0.9293, Val Loss: 0.2346, Val Acc:
0.9167
Epoch 3: Train Loss: 0.1207, Train Acc: 0.9644, Val Loss: 0.1822, Val Acc:
0.9367
Epoch 4: Train Loss: 0.0652, Train Acc: 0.9811, Val Loss: 0.2022, Val Acc:
0.9367
Epoch 5: Train Loss: 0.0323, Train Acc: 0.9902, Val Loss: 0.2170, Val Acc:
0.9350
Epoch 6: Train Loss: 0.0358, Train Acc: 0.9876, Val Loss: 0.2501, Val Acc:
0.9317
Epoch 7: Train Loss: 0.0205, Train Acc: 0.9937, Val Loss: 0.2440, Val Acc:
0.9467
Epoch 8: Train Loss: 0.0073, Train Acc: 0.9980, Val Loss: 0.2113, Val Acc:
0.9467
Epoch 9: Train Loss: 0.0048, Train Acc: 0.9989, Val Loss: 0.2141, Val Acc:
0.9467
Epoch 10: Train Loss: 0.0031, Train Acc: 0.9993, Val Loss: 0.2185, Val Acc:
0.9467
Epoch 11: Train Loss: 0.0008, Train Acc: 1.0000, Val Loss: 0.2110, Val Acc:
0.9500
Epoch 12: Train Loss: 0.0004, Train Acc: 1.0000, Val Loss: 0.2112, Val Acc:
0.9533
Epoch 13: Train Loss: 0.0004, Train Acc: 1.0000, Val Loss: 0.2123, Val Acc:
0.9567
Epoch 14: Train Loss: 0.0003, Train Acc: 1.0000, Val Loss: 0.2135, Val Acc:
0.9567
Epoch 15: Train Loss: 0.0003, Train Acc: 1.0000, Val Loss: 0.2146, Val Acc:
0.9567
Epoch 16: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.2156, Val Acc:
0.9583
Epoch 17: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.2166, Val Acc:
0.9583
Epoch 18: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.2175, Val Acc:
0.9583
Epoch 19: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.2184, Val Acc:
0.9583
Epoch 20: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.2193, Val Acc:
0.9583
Epoch 21: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.2201, Val Acc:
0.9600
Epoch 22: Train Loss: 0.0002, Train Acc: 1.0000, Val Loss: 0.2209, Val Acc:
0.9600
Epoch 23: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2217, Val Acc:
0.9600
Epoch 24: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2224, Val Acc:
0.9600
Epoch 25: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2231, Val Acc:
0.9600
```

```
Epoch 26: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2238, Val Acc:
0.9600
Epoch 27: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2244, Val Acc:
0.9600
Epoch 28: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2250, Val Acc:
0.9600
Epoch 29: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2256, Val Acc:
0.9600
Epoch 30: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2262, Val Acc:
0.9600
Epoch 31: Train Loss: 0.0001, Train Acc: 1.0000, Val Loss: 0.2268, Val Acc:
0.9600
Early stopping triggered
MyModel(
  (network): Sequential(
    (0): Linear(in_features=784, out_features=500, bias=True)
    (1): ReLU()
    (2): Linear(in_features=500, out_features=500, bias=True)
    (3): ReLU()
    (4): Linear(in_features=500, out_features=10, bias=True)
  )
)
Param: network.0.weight Shape: [500, 784]       Numel: 392000
Param: network.0.bias   Shape: [500]    Numel: 500
Param: network.2.weight Shape: [500, 500]       Numel: 250000
Param: network.2.bias   Shape: [500]    Numel: 500
Param: network.4.weight Shape: [10, 500]        Numel: 5000
Param: network.4.bias   Shape: [10]     Numel: 10
Total Numel = 648010
test accuracy: 0.9454
```



24 Comparison on MNIST

- Performance is saturated.
  Type
  No.Layers
  Architecture
  No.Parameters
  Test Accuracy
  LR
  1
  output(10)
  7,850
  0.8813
  MLP
  2
  ReLu(50), output(10)
  39,760
  0.9395
  MLP
  2
  Relu(200), output(10)
  159,010

0.9464
MLP
2
Relu(1000), output(10)
795,010
0.9482
MLP
3
ReLu(500), Relu(500), output(10)
648,010
0.9463

25 Summary

- **Different types of neural networks**
  - *Perceptron* - single node (similar to logistic regression)
  - *Multi-layer perceptron (MLP)* - collection of perceptrons in layers
    - also called *fully-connected layers* or *dense layers*
- **Training**
  - optimize loss function using stochastic gradient descent
- **Advantages**
  - lots of parameters - large capacity to learn from large amounts of data
- **Disadvantages**
  - lots of parameters - easy to overfit data
    - need to monitor the training process
  - sensitive to initialization, learning rate, training algorithm.

26 Other things

- **Numerical stability**
  - normalize the inputs to [-1,1] or [0,1]
- **Improving speed**
  - parallelize computations using GPU (Nvidia+CUDA)
- **Initialization**
  - the resulting network is still sensitive to initialization.
    - Solution: train several networks and combine them as an ensemble.
- **Training problems**
  - For very deep networks, the "vanishing gradient" problem can hinder convergence
  - lack of data
  - We will see how to address these problems later.

27 References

- Software
  - pytorch: https://pytorch.org/
  - see Canvas site for installation tips.

- History:
  - https://en.wikipedia.org/wiki/History_of_artificial_neural_networks
- Online courses:
  - http://cs231n.github.io/neural-networks-1/
  - http://cs231n.github.io/convolutional-networks/